

claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\agent-adffe4196be468c02.jsonl`  
- SHA-256: `38ed0c9c8465910cf4bd62ad822423bea5d29eff900893792fe43457d174ca84`  
- Source modified: `2026-06-16T03:14:30+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `subagents`  
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`
```

Transcript

1. user (2026-06-16T03:12:00.223Z)

Produce a precise inventory of the rally-detection EVALUATION + EXPERIMENT harness code in this repo (cwd = the badminton/sports highlight indexer). This grounds a design doc for strengthening the eval framework, so be exact.

Focus on `backend/eval/` ? read every module there, especially: experiment.py, eval_stats.py, segmentation_metrics.py, score_calibration.py, ablation.py, windowing.py, promotion.py, fusion_golden.py, fusion_compare.py, fusion_audio.py, gt_loader.py, served_gate_a.py (and any others present). Also look for a regression runner (e.g. run_regression.py or similar), the experiment leaderboard (eval_baselines/experiment_leaderboard.json or similar), and how the GOLDEN eval set is stored/loaded (manifest file, gt_loader format, and how many golden videos exist today).

For EACH module report: (1) purpose, (2) key public functions/classes with file:line anchors, (3) what metrics / statistics / splits / calibration it provides, (4) any TODO / gaps / "not wired into default reporting" notes.

Then answer these specific questions precisely:

- How is a "Variant" defined and how does `experiment.compare()` work? What does it output (the "honest" stats block)?
- What metrics exist already: F1@IoU? mAP@tIoU? segment_count_ratio? edit/segmental scores? bootstrap CIs? paired sign-test? calibration (Platt/isotonic, ECE/Brier)?
- How does no-regression / promotion gating work (promotion.py, served_gate_a.py)? What is the eval?serve guard (windowing.py single-sourcing the served preset)?
- What splits are used (LOVO / leave-one-video-out / LOGO / out-of-fold)? Any stacking-leakage guards?
- How are golden labels loaded (gt_loader.py) and what's the on-disk format? How many golden videos / rallies currently?

Return a structured markdown inventory. Distilled, anchor-rich, NO full file dumps. The goal is a complete map of what eval machinery ALREADY EXISTS (so the doc extends it, not reinvents it) and where the gaps are.

2. assistant (2026-06-16T03:12:02.151Z)

I'll systematically inventory the rally-detection eval harness. Let me start by exploring the backend/eval directory structure.

3. user (2026-06-16T03:12:03.705Z)

```
Exit code 127  
/usr/bin/bash: line 1: Get-ChildItem: command not found  
/usr/bin/bash: line 1: Select-Object: command not found  
/usr/bin/bash: line 1: ConvertTo-Csv: command not found
```

4. assistant (2026-06-16T03:12:05.215Z)

Let me use Bash correctly to list the files:

5. user (2026-06-16T03:12:05.798Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\ablation.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\annotations.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\audio\ei_probe.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\audio\__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\batch.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibrate_wasb.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\calibration.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\classifier.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\distill_local.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\eval_partial_labels.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\eval_stats.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\experiment.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\export_wasb_segments.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_audio.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_compare.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\fusion_golden.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gemini_refine.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\gt_loader.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\harness.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\metrics.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\per_user_eval.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\per_user_gate.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\per_user_regression.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\promotion.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\rally_seq_proto.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\regression.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\score_calibration.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\segmentation_metrics.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\served_gate_a.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\serve_contrast.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\_splits.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\training.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval>windowing.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\eval\__init__.py
```

6. assistant (2026-06-16T03:12:07.281Z)

Good. Now let me read the key files. I'll start with the core modules in parallel:

7. user (2026-06-16T03:12:07.954Z)

```
1      """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3      The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
4      a
5      single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
6      training
7      and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
8      ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
9      turns those into an honest summary:
10
11      - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
12      interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
13      many *videos* it wins on (the distribution-free test the repo already leans on:
14      "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
15      noise.
16
17      - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
18      learned
19      producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
```

```

15     match, never a held-out window from the same video).
16
17 See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18 deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19 ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20 """
21 from __future__ import annotations
22
23 import math
24 import random
25 import statistics
26 from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28 Interval = Tuple[float, float]
29 FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32 def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                 n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34     """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36     ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
to
37 resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
band.
38     """
39     vals = [float(v) for v in values]
40     n = len(vals)
41     if n == 0:
42         return (0.0, 0.0)
43     if n == 1:
44         return (vals[0], vals[0])
45     rng = random.Random(seed)
46     means: List[float] = []
47     for _ in range(max(1, n_boot)):
48         means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
49     means.sort()
50     alpha = (1.0 - confidence) / 2.0
51     last = len(means) - 1
52     lo = means[max(0, int(math.floor(alpha * last)))]
53     hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
54     return (lo, hi)
55
56
57 def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
58                 n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
59     """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
(C1)."""
60     vals = [float(v) for v in values]
61     n = len(vals)
62     mean = statistics.mean(vals) if vals else 0.0
63     std = statistics.pstdev(vals) if n > 1 else 0.0
64     lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
65     return {"mean": mean, "std": std, "n": float(n),
66           "ci_low": lo, "ci_high": hi, "confidence": confidence}
67
68
69 def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
70     """Two-sided exact sign test over per-fold deltas (B ? A).
71
72     ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
73     excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
74     distribution-free "did B win on enough *videos*" check.
75     """

```

```

76     wins = sum(1 for d in deltas if d > eps)
77     losses = sum(1 for d in deltas if d < -eps)
78     ties = len(deltas) - wins - losses
79     n_eff = wins + losses
80     if n_eff == 0:
81         p = 1.0
82     else:
83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87           "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),
111     }
112
113 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
115     unit.
116
117     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
118     first-seen (deterministic). Groups with no complement (a single group) yield no
119     fold.
120     """
121     seen: List[Any] = []
122     for g in groups:
123         if g not in seen:
124             seen.append(g)
125     n = len(groups)
126     folds: List[Tuple[List[int], List[int]]] = []
127     for g in seen:
128         test = [i for i in range(n) if groups[i] == g]
129         train = [i for i in range(n) if groups[i] != g]
130         if test and train:
131             folds.append((train, test))
132     return folds
133
134 def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
135 List[Optional[Any]]:
136     """Out-of-fold predictions for honest stacking (C3).
137
138     For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on

```

```

the
136     other groups and returns predictions for ``test_idx`` (in that order); results are
137     scattered back to a list aligned to the input items. So a *learned* producer's
outputs,
138     when used as arbiter features, never include in-sample predictions ? the leakage
path
139     that inflates measured lift on a tiny corpus. Items in a singleton group stay
``None``
140     (needs >= 2 groups to predict).
141     """
142     n = len(groups)
143     oof: List[Optional[Any]] = [None] * n
144     for train_idx, test_idx in grouped_folds(groups):
145         preds = fit_predict(train_idx, test_idx)
146         for pos, ti in enumerate(test_idx):
147             if pos < len(preds):
148                 oof[ti] = preds[pos]
149     return oof
150

```

8. user (2026-06-16T03:12:08.191Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`           ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`         ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`      ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`         ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`        ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19    signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20    given persisted candidate features it produces `List[Interval]` deterministically,
21    with no GPU and zero production risk. So most experiments are pure offline
22    re-scoring of stored data and never touch the served pipeline.
23
24    Three modes (`EXPERIMENT_HARNESS.md` §5):
25    - `compare()`           ? labeled A/B vs golden labels: per-video + LOVO-honest
26      aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27      held-out loop, but variant-parameterised.
28    - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29      variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30      human spot-checks (and what two highlight reels would show side by side).
31    - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32      (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34    Pure + offline + unit-tested. The only DB-touching helper is
35    `load_video_candidates`, which reads persisted candidates via
36    `Database.query_candidate_segments`.
37    """
38    from __future__ import annotations
39
40    import json
41    import logging
42    import os
43    from dataclasses import dataclass, field

```

```

44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats          # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm  # C4: F1@k + segment-count
ratio
56     from backend.eval.score_calibration import fit_isotonic, fit_platt  # C2: calibrate cue
scores
57
58     logger = logging.getLogger(__name__)
59
60     # Where a comparison run appends one reproducible record. Tracked location (NOT under
61     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
62     # regression.DEFAULT_BASELINE_PATH.
63     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
64     _METRICS = ("precision", "recall", "f1", "mean_iou")
65
66
67     class ExperimentError(ValueError):
68         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
69         score an unlabeled video with no pinned model, or a gate referencing an absent
70         feature)."""
71
72
73     # ----- Variant
74
75
76     @dataclass
77     class Variant:
78         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
79
80         A variant turns one video's persisted candidate windows + features into rally
81         intervals. Every knob defaults to the control behaviour (no gate, no learned
82         filter), so a variant that merely carries a new, disabled cue is byte-identical
83         to control ? the core no-regression guarantee.
84
85         Fields:
86         - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87           for the leaderboard and for selecting the served path (the existing
88           ``segmenter_registry`` + ``IndexingConfig`` do the actual selection in production).
89           New cues are recorded here default-OFF.
90         - ``min_crossings`` ? decision-gate Gate A: keep only windows whose persisted
91           ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92           ``rally_gate.py`` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93           windows ? see ``MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94         - ``min_players`` ? decision-gate Gate B (the future person cue): keep windows
95           whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96           cue persists that feature).
97         - ``classifier_model`` ? path to a frozen ``LogisticRegression`` JSON. When set,
98           ``compare()`` scores videos with the SHIPPED model as-is (no per-fold training);
99           required for ``compare_unlabeled`` on a learned variant.
100        - ``feature_names`` ? the persisted features the learned filter consumes. When set
101          WITHOUT ``classifier_model``, ``compare()`` trains a fresh model per LOVO fold
102          (honest) ? the recipe, not a pinned artifact.
103        - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104          overlap-merge gap (seconds), the same ``gemini_refine.merge_overlaps`` default.
105        """

```

```

106
107     name: str
108     segmenter: str = "wasb_hybrid"
109     config_overrides: Dict[str, Any] = field(default_factory=dict)
110     cue_flags: Dict[str, bool] = field(default_factory=dict)
111     min_crossings: int = 0
112     min_players: int = 0
113     classifier_model: Optional[str] = None
114     feature_names: Optional[List[str]] = None
115     threshold: float = 0.5
116     merge_gap: float = 1.0
117     # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118     # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119     # failure where a fixed 0.5 zeroed out a small-candidate video.
120     tune_threshold: bool = False           # pick the F1-best threshold on the train fold
121     calibrate: Optional[str] = None        # None | "platt" | "isotonic" ? calibrate
proba first
122
123     def is_learned(self) -> bool:
124         """True if this variant applies a learned classifier (pinned model or
recipe)."""
125         return self.classifier_model is not None or bool(self.feature_names)
126
127     def to_dict(self) -> dict:
128         return {
129             "name": self.name,
130             "segmenter": self.segmenter,
131             "config_overrides": self.config_overrides,
132             "cue_flags": self.cue_flags,
133             "min_crossings": self.min_crossings,
134             "min_players": self.min_players,
135             "classifier_model": self.classifier_model,
136             "feature_names": self.feature_names,
137             "threshold": self.threshold,
138             "merge_gap": self.merge_gap,
139             "tune_threshold": self.tune_threshold,
140             "calibrate": self.calibrate,
141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
the
151         same, so a regression is always traceable to a recipe change, never to drift."""
152         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
153         return sha1(blob.encode()).hexdigest()[:12]
154
155
156     #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157     #: filter. Always the comparison reference; "rollback" = make this the served variant.
158     CONTROL = Variant(name="control")
159
160
161     # ----- persisted candidates
162
163
164     @dataclass
165     class VideoCandidates:

```

```

166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name -> per-candidate value``, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180     def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181         """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182         ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183         col = vc.features.get(name)
184         n = len(vc.intervals)
185         if col is None:
186             logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                             name, vc.name)
188             return [0.0] * n
189         if len(col) != n:
190             raise ExperimentError(
191                 f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192         return [float(x) for x in col]
193
194
195     def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
196     List[List[float]]:
197         cols = [_feature_column(vc, name) for name in feature_names]
198         return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
199
200     def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201         """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202         players)."""
203         n = len(vc.intervals)
204         mask = [True] * n
205         if variant.min_crossings > 0:
206             col = _feature_column(vc, "net_crossings")
207             mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
208         if variant.min_players > 0:
209             col = _feature_column(vc, "players_in_court")
210             mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
211         return mask
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted ``LogisticRegression``) is supplied by ``compare`` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:

```

```

228         if model is None:
229             if variant.classifier_model is None:
230                 raise ExperimentError(
231                     f"variant {variant.name!r} is learned but has no model to predict "
232                     f"with (pass a fitted model or set classifier_model)")
233                 model = LogisticRegression.load(variant.classifier_model)
234             proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
variant.feature_names))]
235             if calibrator is not None:
236                 proba = [calibrator(p) for p in proba]
237                 thr = variant.threshold if threshold is None else threshold
238                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
239             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
240             return merge_overlaps(kept, gap_tol=variant.merge_gap)
241
242
243     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
244                           train_videos: Sequence[VideoCandidates], iou: float
245                           ) -> "tuple[Optional[Callable[[float], float]],
Optional[float]]":
246         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
247         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
248         (use the variant default). Honest: never looks at the held-out video.
249         """
250         calibrator: Optional[Callable[[float], float]] = None
251         if variant.calibrate:
252             raw: List[float] = []
253             y: List[int] = []
254             for vc in train_videos:
255                 raw.extend(float(p) for p in
256                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
[])))
257                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
258             if variant.calibrate == "platt":
259                 calibrator = fit_platt(raw, y)
260             elif variant.calibrate == "isotonic":
261                 calibrator = fit_isotonic(raw, y)
262             else:
263                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
'platt'/'isotonic')")
264             threshold: Optional[float] = None
265             if variant.tune_threshold:
266                 best_t, best_f1 = variant.threshold, -1.0
267                 for k in range(1, 20):
268                     t = round(0.05 * k, 2)
269                     f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                                vc.gts or [], iou).f1 for vc in train_videos]
271                     mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                     if mean_f1 > best_f1:
273                         best_f1, best_t = mean_f1, t
274                 threshold = best_t
275             return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []

```

```

285     y: List[int] = []
286     for vc in videos:
287         if vc.gts is None:
288             raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289         X.extend(_feature_matrix(vc, variant.feature_names or []))
290         y.extend(label_candidates(vc.intervals, vc.gts, iou))
291     if not X:
292         raise ExperimentError("cannot train a learned variant on zero candidates")
293     return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296 def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                     iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298     """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299     mean aggregate.
300
301     Prediction policy:
302     - **static variant** (no learned filter): predict each video directly ? no
303     training,
304         so no leakage; per-video scoring is already honest.
305     - **learned, pinned model** (`classifier_model` set): score every video with the
306     SHIPPED model. ? optimistic if those videos were in its training set ? use this
307     to
308         report "how the shipped artifact does here", not for the promotion decision.
309     - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
310     ?
311         train on the OTHER videos, predict the held-out one. The number to trust.
312     - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
313     only).
314
315     """
316     for vc in videos:
317         if vc.gts is None:
318             raise ExperimentError(f"{vc.name} has no golden labels; use
319     compare_unlabeled")
320
321     per_video: List[Dict[str, Any]] = []
322     predictions: Dict[str, List[Interval]] = {}
323
324     recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
325     pinned_model = (LogisticRegression.load(variant.classifier_model)
326                     if variant.classifier_model is not None else None)
327     all_model = None
328     if recipe_lovo and not honest:
329         all_model = _train(variant, videos, iou)
330
331     for i, vc in enumerate(videos):
332         cal: Optional[Callable[[float], float]] = None
333         thr: Optional[float] = None
334         if recipe_lovo and honest:
335             others = [v for j, v in enumerate(videos) if j != i]
336             if not others:
337                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
338     pinned model")
339             model: Optional[LogisticRegression] = _train(variant, others, iou)
340             if variant.tune_threshold or variant.calibrate: # operating point from
341     the train fold
342                 assert model is not None # _train never returns
343     None
344                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
345             elif recipe_lovo: # honest=False ? one model over all
346     videos
347                 model = all_model
348             else: # static variant or pinned model
349                 model = pinned_model
350             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)

```

```

341         assert vc.gts is not None                # guarded at function top
342         sc = score(preds, vc.gts, iou)
343         per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344         predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])
375             ratios.append(_segm.segment_count_ratio(preds, gts))
376
377         def _mean(xs: List[float]) -> float:
378             return sum(xs) / len(xs) if xs else 0.0
379         out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380         out["segment_count_ratio"] = _mean(ratios)
381         return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),

```

```

396         "variant_b_ci": _ci_block(eval_b),
397         "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398         "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                          "variant_b": _segmentation_block(videos, eval_b)},
400     }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425
426         by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427         per_video_delta = [
428             {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429             for p in eval_b["per_video"]
430         ]
431
432         # One fingerprint over the whole label set ? detects "the deltas were measured
against
433         # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434         all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436         result: Dict[str, Any] = {
437             "iou": iou,
438             "label_set_hash": gt_hash(all_gts),
439             "num_videos": len(videos),
440             "variant_a": eval_a,
441             "variant_b": eval_b,
442             "delta": delta,
443             "delta_pct": delta_pct,
444             "per_video_delta": per_video_delta,
445         }
446         if honest_stats:
447             result["honest"] = honest_summary(videos, eval_a, eval_b)
448         return result

```

```

449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455                      agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471
472     """
473     preds_a = predict(variant_a, video)
474     preds_b = predict(variant_b, video)
475     mr = match_intervals(preds_a, preds_b, agree_iou)
476
477     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
478              for m in mr.matches]
479     agree_ious = [m.iou for m in mr.matches]
480     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
481     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
482
483     def _dur(ivs: List[Interval]) -> float:
484         return sum(e - s for s, e in ivs)
485
486     return {
487         "video": video.name,
488         "agree_iou": agree_iou,
489         "variant_a": variant_a.name,
490         "variant_b": variant_b.name,
491         "num_a": len(preds_a),
492         "num_b": len(preds_b),
493         "num_agreed": len(agreed),
494         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
495         "duration_a": _dur(preds_a),
496         "duration_b": _dur(preds_b),
497         "agreed": agreed,
498         "a_only": a_only,
499         "b_only": b_only,
500     }
501
502 # ----- leaderboard / DB
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),

```

```

513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                       "spec_hash": result["variant_a"]["spec_hash"],
518                       "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                       "spec_hash": result["variant_b"]["spec_hash"],
521                       "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
the
525     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
526     honest = result.get("honest") or {}
527     if honest.get("paired_delta_f1"):
528         row["honest_delta_f1"] = honest["paired_delta_f1"]
529     entries: List[Dict[str, Any]] = []
530     if os.path.isfile(path):
531         with open(path, encoding="utf-8") as f:
532             entries = json.load(f)
533     entries.append(row)
534     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
535     with open(path, "w", encoding="utf-8") as f:
536         json.dump(entries, f, indent=2)
537     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
538               row["variant_a"]["name"], row["variant_b"]["name"],
539               (result.get("delta") or {}).get("f1", 0.0), path)
540     return row
541
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``
550         JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
551         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
552         but works against any `Database` (including an in-memory one in tests).
553         """
554         rows = db.query_candidate_segments(video_id, detector=detector)
555         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
556         features: Dict[str, List[float]] = {}
557         for name in feature_names:
558             if name == "duration":
559                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
560             else:
561                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
562         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)

```

9. user (2026-06-16T03:12:08.163Z)

```

1     """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3     Temporal IoU (``metrics.score``) is nearly blind to *fragmentation*: a single rally

```

```

4     chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5     still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6     literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7     thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8     sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9     tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10    structure invisibly. See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C4.
11
12    This is **additive**: new functions over the existing ``metrics`` matcher; nothing here
13    changes ``score``/``compare``/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14    ?
15
16    so it runs in the GPU-free / numpy-free lanes too.
17
18    Conventions match ``metrics``: an ``Interval`` is a ``(start, end)`` tuple of seconds.
19    A ``ScoredInterval`` is ``(interval, confidence)`` ? the confidence is whatever the
20    scorer
21    emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
22
23    Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
24    the
25    ordered sequence of segment *labels* and is degenerate for a single action class (every
26    rally carries the same label), so it is intentionally omitted ? F1@k + mAP@tIoU + the
27    segment-count ratio are the over-segmentation-sensitive metrics for this binary setting.
28    """
29    from __future__ import annotations
30
31    from typing import Dict, List, Sequence, Tuple
32
33    from backend.eval.metrics import Interval, interval_iou, score
34
35    #: A predicted interval plus the confidence used to rank it for average precision.
36    ScoredInterval = Tuple[Interval, float]
37
38    #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
39    DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
40    #: Default tIoU sweep for mean average precision ? the action-detection convention.
41    DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
42
43    def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
44                      overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
45        """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.
46
47        Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
48        for 1 rally) scores extra false positives and a merged prediction misses a rally ?
49        both drag F1 down where temporal-IoU alone would not flinch.
50        """
51        return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
52
53    def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
54                        iou_threshold: float = 0.5) -> float:
55        """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
56
57        Predictions are ranked by descending confidence; each is a true positive iff it
58        claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
59        available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
60        there are no ground-truth rallies or no predictions reach the threshold.
61        """
62        n_gt = len(gts)
63        if n_gt == 0 or not scored_preds:
64            return 0.0
65
66        order = sorted(range(len(scored_preds)),
67                      key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by

```

```

index
66     matched_gt = set()
67     precisions: List[float] = []
68     recalls: List[float] = []
69     cum_tp = 0
70     cum_fp = 0
71     for i in order:
72         interval, _conf = scored_preds[i]
73         best_iou, best_gi = 0.0, -1
74         for gi, g in enumerate(gts):
75             if gi in matched_gt:
76                 continue
77             iou = interval_iou(interval, g)
78             if iou >= iou_threshold and iou > best_iou:
79                 best_iou, best_gi = iou, gi
80         if best_gi >= 0:
81             matched_gt.add(best_gi)
82             cum_tp += 1
83         else:
84             cum_fp += 1
85         precisions.append(cum_tp / (cum_tp + cum_fp))
86         recalls.append(cum_tp / n_gt)
87
88     # Monotone precision envelope (take the max precision to the right), then integrate
89     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
90     for i in range(len(precisions) - 2, -1, -1):
91         precisions[i] = max(precisions[i], precisions[i + 1])
92     ap = 0.0
93     prev_recall = 0.0
94     for p, r in zip(precisions, recalls):
95         if r > prev_recall:
96             ap += p * (r - prev_recall)
97             prev_recall = r
98     return ap
99
100
101     def mean_average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
102                               iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
103         """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
104         tious = list(iou_thresholds)
105         if not tious:
106             return 0.0
107         return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
108
109
110     def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
111         """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
112
113         ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
114         merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
115         ``gts``
116         is empty (use F1@k / mAP for the graded picture).
117         """
118         if not gts:
119             return 0.0
120         return len(preds) / len(gts)
121
122     def segmentation_report(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
123                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
124                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
125 object]:
126         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
127
128         ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a

```

```

128     plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result
129     or land in the experiment leaderboard.
130     """
131     preds = [iv for iv, _conf in scored_preds]
132     return {
133         "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()}},
134         "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
135         "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
136         "segment_count_ratio": segment_count_ratio(preds, gts),
137         "num_pred": len(preds),
138         "num_gt": len(gts),
139     }
140

```

10. user (2026-06-16T03:12:08.345Z)

```

1     """Per-cue score calibration (course-correction C2) ? make ``confidence`` mean
something.
2
3     A CNN softmax, an audio onset score, and an integer net-crossing count live on wildly
4     different scales. Fed raw into the learned arbiter (the ADR-004 logistic regression),
the
5     most over-confident cue dominates and starves complementary cues. Calibrating each
6     producer's score to a probability *before* it becomes a feature column makes the
7     ``(value, confidence)`` semantics real and the arbiter's weights stable. See
8     ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C2.
9
10    Two calibrators, both standard:
11    - **Platt scaling** ? fit ``p = sigmoid(a*s + b)`` (parametric; good for monotone,
12      roughly-sigmoidal cues; uses Platt's target smoothing to avoid 0/1 overfitting).
13    - **Isotonic regression** (pool-adjacent-violators) ? non-parametric monotone fit; more
14      flexible, needs a bit more data.
15
16    Plus ``brier_score`` and ``expected_calibration_error`` to *measure* calibration. All
17    **additive**, pure (stdlib only), deterministic ? fits run in the numpy-free lanes too.
18    Calibrators are tiny dataclasses (their params are JSON-friendly), so a fitted
calibrator
19    can be pinned alongside a variant the way the classifier is.
20    """
21    from __future__ import annotations
22
23    import bisect
24    import math
25    from dataclasses import dataclass
26    from typing import List, Sequence, Tuple
27
28    _EPS = 1e-12
29
30
31    def _sigmoid(z: float) -> float:
32        if z >= 0:
33            return 1.0 / (1.0 + math.exp(-z))
34        e = math.exp(z)
35        return e / (1.0 + e)
36
37
38    def _clip01(p: float) -> float:
39        return min(1.0 - _EPS, max(_EPS, p))
40
41
42    @dataclass(frozen=True)
43    class PlattCalibrator:
44        """``p = sigmoid(a*score + b)``. Call it on a raw cue score to get a probability."""

```

```

45     a: float
46     b: float
47
48     def __call__(self, s: float) -> float:
49         return _clip01(_sigmoid(self.a * s + self.b))
50
51
52 @dataclass(frozen=True)
53 class IsotonicCalibrator:
54     """Piecewise-constant monotone map from raw score to probability (PAVA fit).
55
56     ``xs`` are sorted training scores, ``ys`` the non-decreasing fitted probabilities;
57     a query returns the fitted value of the largest ``x <= score`` (clipped at the
ends).
58     """
59     xs: List[float]
60     ys: List[float]
61
62     def __call__(self, s: float) -> float:
63         if not self.xs:
64             return 0.5
65         idx = bisect.bisect_right(self.xs, s) - 1
66         if idx < 0:
67             idx = 0
68         return self.ys[idx]
69
70
71 def fit_platt(scores: Sequence[float], labels: Sequence[int],
72             iters: int = 200, lr: float = 0.5) -> PlattCalibrator:
73     """Fit Platt scaling by gradient descent on (smoothed) log-loss.
74
75     Scores are standardised internally for a stable fit, then folded back so the
returned
76     ``a``/``b`` apply to the **raw** score. Platt target smoothing
77     (``t+ = (N+ + 1)/(N+ + 2)``, ``t- = 1/(N- + 2)``) keeps the fit off the 0/1 rails.
78     Degenerate inputs (empty, zero-variance, single class) fall back to a sane constant
map.
79     """
80     n = len(scores)
81     if n == 0 or n != len(labels):
82         return PlattCalibrator(a=0.0, b=0.0) # ? p = 0.5 everywhere
83     mean = sum(scores) / n
84     var = sum((s - mean) ** 2 for s in scores) / n
85     sd = math.sqrt(var)
86     n_pos = sum(1 for y in labels if y >= 0.5)
87     n_neg = n - n_pos
88     if sd <= _EPS or n_pos == 0 or n_neg == 0:
89         # No usable score spread or one class: best constant probability = base rate.
90         base = _clip01(n_pos / n)
91         return PlattCalibrator(a=0.0, b=math.log(base / (1.0 - base)))
92     t_pos = (n_pos + 1.0) / (n_pos + 2.0)
93     t_neg = 1.0 / (n_neg + 2.0)
94     z = [(s - mean) / sd for s in scores]
95     targets = [t_pos if y >= 0.5 else t_neg for y in labels]
96
97     a, b = 1.0, 0.0 # in standardised space
98     for _ in range(max(1, iters)):
99         ga = gb = 0.0
100         for zi, t in zip(z, targets):
101             p = _sigmoid(a * zi + b)
102             d = p - t
103             ga += d * zi
104             gb += d
105         a -= lr * ga / n
106         b -= lr * gb / n

```

```

107         # Fold standardisation back to raw score:  $z = (s - \text{mean}) / \text{sd}$  ?  $a * z + b = (a / \text{sd}) * s + (b - a * \text{mean} / \text{sd})$ .
108         return PlattCalibrator(a=a / sd, b=b - a * mean / sd)
109
110
111     def _pava(ys: Sequence[float]) -> List[float]:
112         """Pool-adjacent-violators: nearest non-decreasing fit to ``ys`` (in given
113         order)."""
114         blocks: List[List[float]] = [] # each [value, weight]
115         for y in ys:
116             blocks.append([float(y), 1.0])
117             while len(blocks) >= 2 and blocks[-2][0] > blocks[-1][0]:
118                 v2, w2 = blocks.pop()
119                 v1, w1 = blocks.pop()
120                 blocks.append([(v1 * w1 + v2 * w2) / (w1 + w2), w1 + w2])
121         fitted: List[float] = []
122         for value, weight in blocks:
123             fitted.extend([value] * int(round(weight)))
124         return fitted
125
126     def fit_isotonic(scores: Sequence[float], labels: Sequence[int]) -> IsotonicCalibrator:
127         """Fit isotonic (monotone) calibration via PAVA on score-sorted labels."""
128         n = len(scores)
129         if n == 0 or n != len(labels):
130             return IsotonicCalibrator(xs=[], ys=[])
131         order = sorted(range(n), key=lambda i: (scores[i], i))
132         xs = [float(scores[i]) for i in order]
133         ys = [_clip01(v) for v in _pava([float(labels[i]) for i in order])]
134         return IsotonicCalibrator(xs=xs, ys=ys)
135
136
137     def brier_score(probs: Sequence[float], labels: Sequence[int]) -> float:
138         """Mean squared error of probabilities vs 0/1 labels (lower = better calibrated)."""
139         n = len(probs)
140         if n == 0 or n != len(labels):
141             return 0.0
142         return sum((p - y) ** 2 for p, y in zip(probs, labels)) / n
143
144
145     def expected_calibration_error(probs: Sequence[float], labels: Sequence[int],
146                                   n_bins: int = 10) -> float:
147         """Expected calibration error: |accuracy - confidence| per equal-width bin,
148         weighted."""
149         n = len(probs)
150         if n == 0 or n != len(labels):
151             return 0.0
152         bins = max(1, n_bins)
153         bin_sum_p = [0.0] * bins
154         bin_sum_y = [0.0] * bins
155         bin_cnt = [0] * bins
156         for p, y in zip(probs, labels):
157             b = min(bins - 1, int(p * bins))
158             bin_sum_p[b] += p
159             bin_sum_y[b] += y
160             bin_cnt[b] += 1
161         ece = 0.0
162         for i in range(bins):
163             if bin_cnt[i]:
164                 conf = bin_sum_p[i] / bin_cnt[i]
165                 acc = bin_sum_y[i] / bin_cnt[i]
166                 ece += (bin_cnt[i] / n) * abs(acc - conf)
167         return ece
168

```

```

169     def calibration_report(probs: Sequence[float], labels: Sequence[int],
170                           n_bins: int = 10) -> Tuple[float, float]:
171         """Convenience: ``(brier, ece)`` for a set of probabilities vs labels."""
172         return brier_score(probs, labels), expected_calibration_error(probs, labels, n_bins)
173

```

11. assistant (2026-06-16T03:12:11.042Z)

Now let me read the remaining key modules:

12. user (2026-06-16T03:12:11.987Z)

```

1     """Leave-one-signal-out / leave-one-group-out ablation runner (Phase 0).
2
3     A *systematic, repeatable* version of the one-off ablation
4     (``scratch/gate_ab_ablation.py``):
5     run it and it EXCLUDES a single signal, measures whether quality dropped, and reports ?
6     per
7     signal ? the **marginal contribution** ?F1 (full set vs full-minus-that-signal) with a
8     bootstrap
9     95% CI + paired sign test + the C4 over-segmentation ratio. See
10    ``scratch/ABLATION_FRAMEWORK_PLAN.md``
11    (the design) and ``docs/ABLATION_LAYER.md`` (the shipped note).
12
13    **Nothing here is new scoring/stats math.** It is a *pure driver* over the shipped
14    ``experiment.compare`` + ``eval_stats`` engine, exactly as ``compare`` was a driver over
15    ``metrics``/``calibration``. For each signal ``s`` the marginal contribution is:
16
17    ?(s) = score(full set S) - score(S \ {s})          # leave-one-out drop, ?F1
18
19    primary
20
21    operationally ``experiment.compare(videos, variant_a=without_s, variant_b=full)`` ? A is
22    the
23    ablated (smaller) variant, B is full, so the reported B?A delta IS ?(s), a per-video-
24    paired ?F1
25    with bootstrap CI + exact sign test, LOVO-honest for learned variants.
26
27    Two ablation granularities fall out of one inventory:
28
29    - **Leave-one-signal-out (LOSO)** ? remove ONE column at a time (fine-grained).
30    - **Leave-one-GROUP-out (LOGO)** ? remove a whole producer's columns at once (the
31    owner's
32    "is this old SIGNAL still relevant?" question at cue granularity).
33
34    A signal can be a **feature column** (person/audio/shuttle columns the learned scorer
35    weights) or
36    a **gate knob** (Gate-A's ``min_crossings`` threshold ? ablated by zeroing the knob, NOT
37    by
38    dropping a column). The runner handles both uniformly: ``without_s`` subtracts the
39    columns AND/OR
40    zeroes the gate knob from ``cfg.full``.
41
42    Honest small-N contract (inherited from ``eval_stats`` / ``per_user_gate``; S6 of the
43    plan):
44
45    - never a bare mean ? every row ships a CI + the exact sign test;
46    - a signal **earns its keep** only when removal drops F1 with **CI excluding 0 AND the
47    sign test
48    agreeing** (the ``per_user_gate`` bar, in reverse);
49    - **structural signals are exempt from auto-demotion** ? a guard whose value is in
50    failure modes
51
52    the golden corpus may lack (Gate-A vs held-shuttle FPs) is marked ``structural`` and
53    is NEVER
54    flagged dead weight on "no measured lift"; its row reports the (possibly ?0) golden
55    ? honestly
56    but tags ``structural_protected``;

```

```

38         - the runner emits ``earns_keep`` / ``suggestive`` / ``inert`` /
``structural_protected`` ? it
39         MUST NOT print "irrelevant" or "proven useless" (one wash at n=6 is noise, not
proof).
40
41         Pure + offline + GPU-free. Reuses the persisted golden feature substrate
(``fusion_golden.py``
42         writes one ``<name>_golden_features.json`` per video). CLI::
43
44         python -m backend.eval.ablation --features-dir output [--pdf out.pdf]
45         """
46         from __future__ import annotations
47
48         import argparse
49         import glob
50         import json
51         import logging
52         import os
53         from dataclasses import dataclass, field
54         from typing import Any, Dict, List, Optional, Sequence, Tuple
55
56         from backend.eval import experiment
57         from backend.eval.experiment import Variant, VideoCandidates
58
59         logger = logging.getLogger(__name__)
60
61         # --- The signal inventory (the things the runner ablates), grouped by producer.
62         # Source: distill_local.FEATURE_NAMES (shuttle, includes net_crossings), fusion_features
63         # (person), audio_features (audio). The shuttle group's velocity block + duration are
the
64         # learned-column shuttle signals; net_crossings is BOTH a column and the Gate-A gate
knob.
65         SHUTTLE_COLUMNS: Tuple[str, ...] = ("duration", "peak_velocity", "mean_velocity",
66                                             "active_ratio", "net_crossings")
67         PERSON_COLUMNS: Tuple[str, ...] = ("players_in_court", "two_sided_motion",
"mean_player_motion",
68                                             "in_court_ratio", "shuttle_player_colocation")
69         AUDIO_COLUMNS: Tuple[str, ...] = ("audio_hit_count", "audio_hit_rate",
"audio_hit_strength_mean")
70
71         # Verdict vocabulary (the no-over-claim contract ? never "irrelevant"/"proven useless").
72         EARNS_KEEP = "earns_keep" # CI clears 0 AND sign test agrees ? removing it
hurts
73         SUGGESTIVE = "suggestive" # point estimate leans positive but CI spans 0
74         INERT = "inert" # wash this run (?0, CI spans 0) ? candidate,
not proof
75         STRUCTURAL_PROTECTED = "structural_protected" # a guard; never auto-demoted on "no
lift"
76
77         #: Human-readable one-liners for each verdict (used in the PDF + table legend).
78         VERDICT_LABEL = {
79             EARNS_KEEP: "clears-bar",
80             SUGGESTIVE: "suggestive",
81             INERT: "inert",
82             STRUCTURAL_PROTECTED: "structural-protected",
83         }
84
85
86         @dataclass(frozen=True)
87         class Signal:
88             """One unit of ablation ? a feature column (or a named group) and/or a gate knob.
89
90             - ``name`` ? table/row identity ("person.colocation" | "gate_a" | "audio").
91             - ``group`` ? producer ("person" | "gate_a" | "audio" | "shuttle"). LOGO ablates a

```

```

whole group.
92     - ``columns`` ? the persisted feature columns this signal contributes (may be empty
for a
93     pure gate signal like Gate-A whose column is shared with the shuttle group).
94     - ``gate_knob`` ? e.g. ``"min_crossings"`` for the Gate-A threshold; ablated by
zeroing it on
95     the full variant rather than dropping a column.
96     - ``structural`` ? a guard whose value is in failure modes the golden corpus may
lack; NEVER
97     flagged dead weight on "no measured lift" (mirrors ``per_user_gate``
``structural=True``).
98     ""
99
100     name: str
101     group: str
102     columns: Tuple[str, ...] = ()
103     gate_knob: Optional[str] = None
104     structural: bool = False
105
106
107     @dataclass
108     class AblationConfig:
109         """How to run an ablation over a corpus.
110
111         - ``full`` ? the full-set reference variant (B). For the litmus Gate-A reproduction
this is
112         the STATIC ``Variant(min_crossings=2)`` (no classifier); for a learned ablation it
carries
113         ``feature_names`` = all columns and the gates ON.
114         - ``signals`` ? the inventory to ablate.
115         - ``granularity`` ? ``"signal"`` (LOSO, one column-set at a time) or ``"group"``
(LOGO).
116         - ``iou`` ? temporal-IoU match threshold.
117         - ``honest`` ? LOVO for learned variants (the number to trust).
118         - ``seed`` ? threaded into the ``eval_stats`` bootstrap (determinism).
119         - ``p_threshold`` ? sign-test significance bar for the ``earns_keep`` verdict.
120         ""
121
122         full: Variant
123         signals: List[Signal]
124         granularity: str = "group"
125         iou: float = 0.5
126         honest: bool = True
127         seed: int = 0
128         p_threshold: float = 0.05
129
130
131     @dataclass
132     class AblationRow:
133         """One signal's marginal-contribution row (?s) + CI + sign test + C4 + a
verdict)."""
134
135         signal: str
136         group: str
137         structural: bool
138         dropped_columns: List[str]
139         gate_knob: Optional[str]
140         n: int
141         delta_f1: float
142         ci_low: float
143         ci_high: float
144         ci_excludes_zero: bool
145         sign_wins: int
146         sign_losses: int
147         sign_p_value: float

```

```

148     f1_full: float
149     f1_ablated: float
150     seg_ratio_full: float
151     seg_ratio_ablated: float
152     verdict: str
153     structural_protected: bool
154
155     def as_dict(self) -> Dict[str, Any]:
156         return {
157             "signal": self.signal,
158             "group": self.group,
159             "structural": self.structural,
160             "dropped_columns": list(self.dropped_columns),
161             "gate_knob": self.gate_knob,
162             "n": self.n,
163             "delta_f1": self.delta_f1,
164             "ci_low": self.ci_low,
165             "ci_high": self.ci_high,
166             "ci_excludes_zero": self.ci_excludes_zero,
167             "sign_test": {"wins": self.sign_wins, "losses": self.sign_losses,
168                           "p_value": self.sign_p_value},
169             "f1_full": self.f1_full,
170             "f1_ablated": self.f1_ablated,
171             "segment_count_ratio_full": self.seg_ratio_full,
172             "segment_count_ratio_ablated": self.seg_ratio_ablated,
173             "verdict": self.verdict,
174             "structural_protected": self.structural_protected,
175         }
176
177
178     @dataclass
179     class AblationReport:
180         """All signals' rows + the run's provenance ? a structured, JSON-able ablation
181         result."""
182         granularity: str
183         iou: float
184         num_videos: int
185         honest_lovo: bool
186         full_set_signals: List[str]
187         full_spec_hash: str
188         label_set_hash: str
189         rows: List[AblationRow] = field(default_factory=list)
190
191         def sorted_rows(self) -> List[AblationRow]:
192             """Rows by marginal ?F1 desc ? the contribution ranking (the plan's table
193             order)."""
194             return sorted(self.rows, key=lambda r: r.delta_f1, reverse=True)
195
196         def as_dict(self) -> Dict[str, Any]:
197             return {
198                 "granularity": self.granularity,
199                 "iou": self.iou,
200                 "num_videos": self.num_videos,
201                 "honest_lovo": self.honest_lovo,
202                 "full_set_signals": list(self.full_set_signals),
203                 "full_spec_hash": self.full_spec_hash,
204                 "label_set_hash": self.label_set_hash,
205                 "rows": [r.as_dict() for r in self.sorted_rows()],
206             }
207
208     # ----- the driver
209
210

```

```

211     def _ablated_variant(full: Variant, columns: Sequence[str], gate_knob: Optional[str]) ->
Variant:
212         """Build the leave-one-out (smaller) variant from ``full``: drop ``columns`` from
its
213         ``feature_names`` and/or zero the named gate knob. Pure ? derives a new Variant,
never
214         mutates ``full``."""
215         cols = set(columns)
216         fn = full.feature_names
217         ablated_fn = [c for c in fn if c not in cols] if fn is not None else fn
218         overrides: Dict[str, Any] = {"feature_names": ablated_fn}
219         if gate_knob == "min_crossings":
220             overrides["min_crossings"] = 0
221         elif gate_knob == "min_players":
222             overrides["min_players"] = 0
223         elif gate_knob is not None:
224             raise ValueError(f"unknown gate_knob {gate_knob!r} (expected
'min_crossings'/'min_players')")
225         dropped_label = ",".join(sorted(cols)) or (gate_knob or "?")
226         return experiment.Variant.from_dict(**full.to_dict(), **overrides,
227                                           "name": f"?{dropped_label}")
228
229
230     def _classify(structural: bool, ci_excludes_zero: bool, sign_wins: int, sign_losses:
int,
231                  sign_p: float, p_threshold: float) -> Tuple[str, bool]:
232         """The honest verdict (no over-claim). Returns ``(verdict, structural_protected)``.
233
234         A structural guard is reported but never demoted on "no lift". Otherwise: removing
the
235         signal *earns its keep* when the ?F1 CI clears 0 AND the sign test agrees (majority
worse on
236         removal, ``p <= p_threshold``); a positive-leaning but CI-spanning row is
``suggestive``; a
237         wash this run is ``inert`` (a candidate, never "proven useless" at n=6)."""
238         if structural:
239             return STRUCTURAL_PROTECTED, True
240         sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
241         if ci_excludes_zero and sign_agrees:
242             return EARNS_KEEP, False
243         if sign_wins > sign_losses:
244             # leans toward "removal hurts" but evidence is
thin
245             return SUGGESTIVE, False
246         return INERT, False
247
248     def ablate_one(videos: Sequence[VideoCandidates], cfg: AblationConfig,
249                  signal: Signal) -> AblationRow:
250         """One ablation row: marginal ?(s) + CI + sign test + C4 + a verdict.
251
252         Drops ``signal``'s columns / zeroes its gate knob from ``cfg.full`` to form the
ablated
253         variant A, then calls ``experiment.compare(A, full)`` so the reported B?A ?F1 IS
?(s).
254         Reuses ``compare`` verbatim ? learned variants stay LOVO-honest, the C1+C4 honest
stats
255         come for free."""
256         ablated = _ablated_variant(cfg.full, signal.columns, signal.gate_knob)
257         res = experiment.compare(videos, ablated, cfg.full, iou=cfg.iou, honest=cfg.honest,
258                                honest_stats=True)
259         # Re-pair the per-video ?F1 with the configured bootstrap seed for determinism (the
default
260         # honest block uses seed=0; thread cfg.seed explicitly so a run is reproducible).
261         from backend.eval.eval_stats import paired_delta_ci
262         by_a = {p["video"]: p["f1"] for p in res["variant_a"]["per_video"]}

```

```

263     by_b = {p["video"]: p["f1"] for p in res["variant_b"]["per_video"]}
264     vids = [p["video"] for p in res["variant_a"]["per_video"]]
265     pd = paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids],
seed=cfg.seed)
266     st = pd["sign_test"]
267     sign_wins, sign_losses = int(st["wins"]), int(st["losses"])
268     sign_p = float(st["p_value"])
269     verdict, protected = _classify(signal.structural, bool(pd["ci_excludes_zero"]),
270                                   sign_wins, sign_losses, sign_p, cfg.p_threshold)
271
272     seg_full = res["honest"]["segmentation"]["variant_b"]["segment_count_ratio"]
273     seg_abl = res["honest"]["segmentation"]["variant_a"]["segment_count_ratio"]
274
275     return AblationRow(
276         signal=signal.name,
277         group=signal.group,
278         structural=signal.structural,
279         dropped_columns=list(signal.columns),
280         gate_knob=signal.gate_knob,
281         n=int(pd["n"]),
282         delta_f1=float(pd["mean_delta"]),
283         ci_low=float(pd["ci_low"]),
284         ci_high=float(pd["ci_high"]),
285         ci_excludes_zero=bool(pd["ci_excludes_zero"]),
286         sign_wins=sign_wins,
287         sign_losses=sign_losses,
288         sign_p_value=sign_p,
289         f1_full=float(res["variant_b"]["aggregate"]["f1"]),
290         f1_ablated=float(res["variant_a"]["aggregate"]["f1"]),
291         seg_ratio_full=float(seg_full),
292         seg_ratio_ablated=float(seg_abl),
293         verdict=verdict,
294         structural_protected=protected,
295     )
296
297
298     def _collapse_to_groups(signals: Sequence[Signal]) -> List[Signal]:
299         """Collapse a LOSO inventory into one Signal per producer GROUP (LOGO): union the
columns,
300         OR the gate knob, OR the structural flag. First-seen group order is preserved."""
301         order: List[str] = []
302         by_group: Dict[str, List[Signal]] = {}
303         for s in signals:
304             if s.group not in by_group:
305                 order.append(s.group)
306                 by_group[s.group] = []
307             by_group[s.group].append(s)
308         out: List[Signal] = []
309         for g in order:
310             members = by_group[g]
311             cols: Tuple[str, ...] = tuple(dict.fromkeys(c for m in members for c in
m.columns))
312             knob = next((m.gate_knob for m in members if m.gate_knob), None)
313             structural = any(m.structural for m in members)
314             out.append(Signal(name=g, group=g, columns=cols, gate_knob=knob,
structural=structural))
315         return out
316
317
318     def run_ablation(videos: Sequence[VideoCandidates], cfg: AblationConfig) ->
AblationReport:
319         """Loop the inventory at the configured granularity ? an ``AblationReport`` (sorted
table).
320
321         A pure driver: each row is one ``experiment.compare(without_s, full)``.

```

```

``granularity="group"``
322         (LOGO) ablates a whole producer's columns at once; ``"signal"`` (LOSO) one column-
set at a
323         time. The full reference is scored once implicitly per row (cheap re-scoring;
detection is
324         already persisted)."""
325         if len(videos) < 1:
326             raise experiment.ExperimentError("run_ablation needs at least one labeled
video")
327         if cfg.granularity not in ("signal", "group"):
328             raise ValueError(f"granularity must be 'signal' or 'group', got
{cfg.granularity!r}")
329
330         inventory = cfg.signals if cfg.granularity == "signal" else
_collapse_to_groups(cfg.signals)
331         rows = [ablate_one(videos, cfg, s) for s in inventory]
332
333         # Provenance: a full-set evaluation gives the honest-LOVO flag + the label-set
fingerprint.
334         full_eval = experiment.evaluate_variant(videos, cfg.full, iou=cfg.iou,
honest=cfg.honest)
335         from backend.eval.regression import gt_hash
336         all_gts = [iv for vc in videos for iv in (vc.gts or [])]
337         return AblationReport(
338             granularity=cfg.granularity,
339             iou=cfg.iou,
340             num_videos=len(videos),
341             honest_lovo=bool(full_eval["honest_lovo"]),
342             full_set_signals=[s.group for s in _collapse_to_groups(cfg.signals)],
343             full_spec_hash=cfg.full.spec_hash(),
344             label_set_hash=gt_hash(all_gts),
345             rows=rows,
346         )
347
348
349     # ----- default
inventories
350
351
352     def default_signals() -> List[Signal]:
353         """The current 4-group inventory (LOSO granularity): shuttle (velocity block +
duration),
354         gate_a (the structural net-crossings gate AND its column), person, audio.
355
356         Gate-A is marked ``structural`` (the held-shuttle FP guard) so it is reported but
never
357         flagged dead weight on "no measured lift". It ablates via the ``min_crossings`` knob
AND by
358         dropping the ``net_crossings`` column, so the leave-one-out genuinely removes the
whole cue."""
359         return [
360             Signal("shuttle.duration", "shuttle", ("duration",)),
361             Signal("shuttle.velocity", "shuttle", ("peak_velocity", "mean_velocity",
"active_ratio")),
362             Signal("gate_a", "gate_a", ("net_crossings",), gate_knob="min_crossings",
structural=True),
363             Signal("person", "person", PERSON_COLUMNS),
364             Signal("audio", "audio", AUDIO_COLUMNS),
365         ]
366
367
368     def static_gate_a_signal() -> Signal:
369         """The Gate-A-only structural signal (a pure gate knob over the recall-oriented
proposal
370         set). Ablating it against the static ``Variant(min_crossings=2)`` full reproduces

```

```

the
371         hand-measured Gate-A result (?F1 ? +0.074, 6/6, p ? 0.031) ? the framework's litmus
test. """
372         return Signal("gate_a", "gate_a", (), gate_knob="min_crossings", structural=True)
373
374
375     def litmus_config() -> AblationConfig:
376         """The Gate-A reproduction config: full = STATIC ``Variant(min_crossings=2)`` (no
learned
377         classifier), inventory = the single Gate-A gate signal. Ablating it = CONTROL+gate
vs CONTROL
378         ? exactly the historical measurement (``scratch/gate_ab_ablation.py``). """
379         full = experiment.Variant(name="control_gate_a", min_crossings=2)
380         return AblationConfig(full=full, signals=[static_gate_a_signal()],
granularity="signal")
381
382
383     # ----- corpus
I/O
384
385
386     def load_videos(features_dir: str) -> List[VideoCandidates]:
387         """Load every ``*_golden_features.json`` into a ``VideoCandidates`` (intervals +
shuttle +
388         person + audio columns + golden gts). Audio columns are optional (older JSONs
predate the
389         audio augment) ? a missing block defaults those columns to 0.0 so the loader never
crashes,
390         mirroring ``experiment._feature_column``'s tolerance. """
391         videos: List[VideoCandidates] = []
392         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
393             with open(path, encoding="utf-8") as f:
394                 d = json.load(f)
395                 cands = d["candidates"]
396                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
397                 features: Dict[str, List[float]] = {}
398                 for fn in SHUTTLE_COLUMNS:
399                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
400                 for fn in PERSON_COLUMNS:
401                     features[fn] = [float(c["person"][fn]) for c in cands]
402                 for fn in AUDIO_COLUMNS:
403                     features[fn] = [float((c.get("audio") or {}).get(fn, 0.0)) for c in cands]
404                 gts = [(float(a), float(b)) for a, b in d["gts"]]
405                 videos.append(VideoCandidates(name=d["name"], intervals=intervals,
406                                             features=features, gts=gts))
407         return videos
408
409
410     def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
411         """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3)
weighted
412         by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the
learned path
413         (distinct from the static Gate-A litmus). """
414         cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
415         return experiment.Variant(name="full_learned", feature_names=cols,
416                                 threshold=threshold, min_crossings=min_crossings)
417
418
419     # ----- table
rendering
420
421
422     def format_table(report: AblationReport) -> str:
423         """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution

```

```

ranking)."""
424     rows = report.sorted_rows()
425     header = (f"Ablation report ? full set = {{{', '.join(report.full_set_signals)}}}"
426               f" . n={report.num_videos} . IoU {report.iou}"
427               f" . {'LOVO-honest' if report.honest_lovo else 'static'}"
428               f" . {report.granularity.upper()}")
429     cols = ["signal", "?F1(full?ablated)", "95% CI", "sign test", "CI?0", "verdict"]
430     widths = [18, 17, 18, 14, 5, 22]
431
432     def _fmt_row(cells: Sequence[str]) -> str:
433         return " ".join(c.ljust(w) for c, w in zip(cells, widths))
434
435     lines = [header, "", _fmt_row(cols), _fmt_row(["-" * w for w in widths])]
436     for r in rows:
437         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
438         sign = f"{r.sign_wins}W/{r.sign_losses}L p={r.sign_p_value:.3f}"
439         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
440         lines.append(_fmt_row([r.signal, f"{r.delta_f1:+.3f}", ci, sign,
441                               "yes" if r.ci_excludes_zero else "no", verdict]))
442     return "\n".join(lines)
443
444
445     # ----- PDF
446 export
447
448     def export_pdf(report: AblationReport, path: str) -> str:
449         """Render the ``AblationReport`` to a clean reportlab PDF: a table of each signal's
450         marginal
451         contribution + CI + sign test + a verdict column (clears-bar / suggestive / inert /
452         structural-protected). Reuses the muted booktabs aesthetic of
453         ``scratch/quality_report.py``.
454
455         Returns the written path. ``reportlab`` is an offline, pure-Python dependency (no
456         GPU)."""
457         from reportlab.lib import colors
458         from reportlab.lib.enums import TA_CENTER, TA_LEFT
459         from reportlab.lib.pagesizes import A4
460         from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
461         from reportlab.lib.units import cm
462         from reportlab.platypus import (Paragraph, SimpleDocTemplate, Spacer, Table,
463         TableStyle)
464
465         navy = colors.HexColor("#1F3A5F")
466         black = colors.HexColor("#000000")
467         darkgrey = colors.HexColor("#333333")
468         light = colors.HexColor("#F0F0F0")
469         hairline = colors.HexColor("#BFBFBF")
470         row_alt = colors.HexColor("#F7F7F7")
471
472         base = getSampleStyleSheet()["Normal"]
473         title = ParagraphStyle("Title2", parent=base, fontName="Helvetica-Bold",
474                                fontSize=16,
475                                leading=20, alignment=TA_CENTER, textColor=navy,
476                                spaceAfter=4)
477         subtitle = ParagraphStyle("Sub", parent=base, fontName="Helvetica-Oblique",
478                                   fontSize=9.5,
479                                   leading=13, alignment=TA_CENTER, textColor=darkgrey,
480                                   spaceAfter=10)
481         h2 = ParagraphStyle("H2", parent=base, fontName="Helvetica-Bold", fontSize=11,
482                              leading=15,
483                              textColor=navy, spaceBefore=10, spaceAfter=4)
484         body = ParagraphStyle("Body", parent=base, fontName="Helvetica", fontSize=9,
485                               leading=12.5,
486                               alignment=TA_LEFT, textColor=black, spaceAfter=5)

```

```

477     cell = ParagraphStyle("Cell", parent=base, fontName="Helvetica", fontSize=8.2,
leading=10.5,
478                             textColor=black)
479     cell_b = ParagraphStyle("CellB", parent=base, fontName="Helvetica-Bold",
fontSize=8.2,
480                             leading=10.5, textColor=black)
481     head = ParagraphStyle("Head", parent=base, fontName="Helvetica-Bold", fontSize=8.2,
482                             leading=10.5, textColor=black)
483
484     rows = report.sorted_rows()
485     story: List[Any] = [
486         Paragraph("Signal Ablation & Feature-Enablement Report", title),
487         Paragraph(
488             f"Leave-one-{'group' if report.granularity == 'group' else 'signal'}-out
marginal "
489             f"contribution &mdash; n={report.num_videos} golden matches, IoU
{report.iou}, "
490             f"{'honest LOVO' if report.honest_lovo else 'static re-score'}. "
491             "Honest measured results only.",
492             subtitle),
493     ]
494
495     full_set = ", ".join(report.full_set_signals)
496     story.append(Paragraph("Run provenance", h2))
497     story.append(Paragraph(
498         f"Full signal set: <b>{{{full_set}}}</b>. Variant spec_hash "
499         f"<font face='Courier'>{report.full_spec_hash}</font>, label-set hash "
500         f"<font face='Courier'>{report.label_set_hash}</font>. Each row is the held-out
"
501         "&Delta;F1 of removing one signal from the full set (full &minus; ablated), with
a "
502         "bootstrap 95% CI and an exact paired sign test &mdash; a signal earns its keep
only "
503         "when removing it drops F1 with the CI clearing 0 <i>and</i> the sign test
agreeing.",
504         body))
505
506     story.append(Paragraph("Marginal contribution per signal", h2))
507     table_data: List[List[Any]] = [[
508         Paragraph("Signal", head), Paragraph("Group", head),
509         Paragraph("&Delta;F1 (full&minus;abl.)", head), Paragraph("95% CI", head),
510         Paragraph("Sign test", head), Paragraph("CI&ne;0", head),
511         Paragraph("Over-seg", head), Paragraph("Verdict", head),
512     ]]
513     for r in rows:
514         ci = f"[{r.ci_low:+.3f}, {r.ci_high:+.3f}]"
515         sign = f"{r.sign_wins}/{r.sign_losses}L p={r.sign_p_value:.3f}"
516         overseg = f"{r.seg_ratio_full:.2f}&rarr;{r.seg_ratio_ablated:.2f}"
517         verdict = VERDICT_LABEL.get(r.verdict, r.verdict)
518         df1_style = cell_b if r.verdict == EARNNS_KEEP else cell
519         table_data.append([
520             Paragraph(r.signal, cell), Paragraph(r.group, cell),
521             Paragraph(f"{r.delta_f1:+.3f}", df1_style), Paragraph(ci, cell),
522             Paragraph(sign, cell), Paragraph("yes" if r.ci_excludes_zero else "no",
cell),
523             Paragraph(overseg, cell), Paragraph(verdict, cell),
524         ])
525     col_widths = [2.7 * cm, 1.7 * cm, 2.4 * cm, 2.7 * cm, 2.6 * cm, 1.1 * cm, 1.9 * cm,
2.6 * cm]
526     tbl = Table(table_data, colWidths=col_widths, repeatRows=1)
527     last = len(table_data) - 1
528     tbl.setStyle(TableStyle([
529         ("BACKGROUND", (0, 0), (-1, 0), light),
530         ("VALIGN", (0, 0), (-1, -1), "MIDDLE"),
531         ("TOPPADDING", (0, 0), (-1, -1), 4),

```

```

532         ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
533         ("LEFTPADDING", (0, 0), (-1, -1), 4),
534         ("RIGHTPADDING", (0, 0), (-1, -1), 4),
535         ("LINEABOVE", (0, 0), (-1, 0), 1.0, black),
536         ("LINEBELOW", (0, 0), (-1, 0), 0.8, black),
537         ("LINEBELOW", (0, 1), (-1, max(1, last - 1)), 0.3, hairline),
538         ("LINEBELOW", (0, last), (-1, last), 1.0, black),
539         ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, row_alt]),
540     ]))
541     story.append(tbl)
542     story.append(Spacer(1, 0.3 * cm))
543
544     story.append(Paragraph("Verdict key", h2))
545     story.append(Paragraph(
546         "<b>clears-bar</b> &mdash; removing the signal drops F1 with the &Delta;F1 CI
clearing 0 "
547         "and the sign test agreeing (the signal earns its keep). "
548         "<b>suggestive</b> &mdash; the point estimate leans toward 'removal hurts' but
the CI "
549         "spans 0 (keep, watch as the corpus grows). "
550         "<b>inert</b> &mdash; removal is a wash this run; a candidate for retirement
only after a "
551         "trend across multiple runs, never on one wash at small n. "
552         "<b>structural-protected</b> &mdash; a guard whose value is in failure modes the
golden "
553         "corpus may lack (e.g. Gate-A vs held-shuttle false positives); reported
honestly but "
554         "never auto-flagged dead weight on 'no measured lift'.",
555         body))
556
557     doc = SimpleDocTemplate(path, pagesize=A4, leftMargin=2 * cm, rightMargin=2 * cm,
558                             topMargin=1.8 * cm, bottomMargin=1.8 * cm,
559                             title="Signal Ablation & Feature-Enablement Report",
560                             author="badminton-highlight-indexer")
561     doc.build(story)
562     logger.info("wrote ablation PDF -> %s", path)
563     return path
564
565
566     # -----
567     CLI
568
569     def main() -> None:
570         ap = argparse.ArgumentParser(
571             description="LOSO/LOGO ablation runner over the golden feature substrate (pure
driver "
572             "over experiment.compare + eval_stats; no new scoring math).")
573         ap.add_argument("--features-dir", default="output",
574             help="dir of *_golden_features.json (the persisted golden
substrate)")
575         ap.add_argument("--granularity", choices=("signal", "group"), default="group",
576             help="leave-one-SIGNAL-out (column) or leave-one-GROUP-out
(producer)")
577         ap.add_argument("--iou", type=float, default=0.5)
578         ap.add_argument("--seed", type=int, default=0, help="bootstrap seed (determinism)")
579         ap.add_argument("--litmus", action="store_true",
580             help="reproduce the static Gate-A measurement (CONTROL vs
min_crossings=2) "
581             "instead of the learned full-set ablation")
582         ap.add_argument("--pdf", default=None, help="also write a PDF ablation report to
this path")
583         ap.add_argument("--json", default=None, help="also write the structured report JSON
here")
584         args = ap.parse_args()

```

```

585
586     videos = load_videos(args.features_dir)
587     if len(videos) < 2:
588         raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} "
589                           f"(found {len(videos)})")
590
591     if args.litmus:
592         cfg = litmus_config()
593         cfg.iou, cfg.seed = args.iou, args.seed
594     else:
595         cfg = AblationConfig(full=learned_full_variant(), signals=default_signals(),
596                               granularity=args.granularity, iou=args.iou, seed=args.seed)
597
598     report = run_ablation(videos, cfg)
599     print(format_table(report))
600
601     if args.json:
602         with open(args.json, "w", encoding="utf-8") as f:
603             json.dump(report.as_dict(), f, indent=2)
604             print(f"\nwrote report JSON -> {args.json}")
605     if args.pdf:
606         export_pdf(report, args.pdf)
607         print(f"wrote PDF report -> {args.pdf}")
608
609
610 if __name__ == "__main__":
611     main()
612

```

13. user (2026-06-16T03:12:11.960Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4     ?
5     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6     built
7     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9     pipeline
10    (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11    merge_gap=2.0,
12    min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set
13    is
14    measured on candidates **production never serves**, so a "win" need not transfer. This
15    is
16    exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
17    set
18    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
19
20    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
21    the
22    SERVED preset is **single-sourced from the same Pydantic config defaults production
23    reads**
24    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
25    Helpers
26    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
27    *actual* served windowing out of a live config (overrides included), so an A/B can
28    target the
29    operating point a given deployment truly serves.
30
31    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
32    windowing* ? lives in :mod:`backend.eval.promotion` (which composes `eval_stats` +
33    `per_user_gate`). This module is the **what windowing** ; that one is the **promotion
34    gate**.

```

```

23
24     Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25     caller opts in (``distill_local`` re-points its module-level presets here, byte-
identically).
26     """
27     from __future__ import annotations
28
29     from dataclasses import dataclass
30     from typing import Any, Dict, List, Mapping, Tuple
31
32     from backend.config.models import IndexingConfig
33
34     Interval = Tuple[float, float]
35
36     __all__ = [
37         "WindowingPreset",
38         "SERVED",
39         "PROPOSAL",
40         "PRESETS",
41         "served_windowing_from_config",
42         "build_windows",
43         "windows_to_intervals",
44     ]
45
46
47     @dataclass(frozen=True)
48     class WindowingPreset:
49         """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51         ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52         "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53         ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54         knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55         """
56
57         name: str
58         velocity_thresh: float
59         merge_gap: float
60         min_window_duration: float
61
62         def as_tuple(self) -> Tuple[float, float, float]:
63             """(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
64             ``trajectory_to_action_windows`` / ``distill_local`` call sites already
speak."""
65             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
66
67         def to_dict(self) -> Dict[str, Any]:
68             return {
69                 "name": self.name,
70                 "velocity_thresh": self.velocity_thresh,
71                 "merge_gap": self.merge_gap,
72                 "min_window_duration": self.min_window_duration,
73             }
74
75
76     def _served_preset_from_defaults() -> WindowingPreset:
77         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.

```

```

78
79     ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
80     - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
81     - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
82     - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)
83
84     Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
85     point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
86     so the two can never silently diverge again.
87     """
88     idx = IndexingConfig() # construct with defaults ? the served operating point
89     return WindowingPreset(
90         name="served",
91         velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
92         merge_gap=float(idx.dead_time_merge_gap),
93         min_window_duration=float(idx.min_action_window_duration),
94     )
95
96
97     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
98     #: only promotable if it holds up HERE ? the operating point real users get.
99     SERVED: WindowingPreset = _served_preset_from_defaults()
100
101     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
102     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
103     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
104     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
105     PROPOSAL: WindowingPreset = WindowingPreset(
106         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
107     )
108
109     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
110     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
111
112
113     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
114         """The SERVED windowing a *specific* live config serves (overrides honoured).
115
116         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
117         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
118         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
119         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
120         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
121         """
122         idx = _as_mapping(config, "indexing")
123         family_cfg = _as_mapping(idx, family)
124         return WindowingPreset(
125             name=f"served:{family}",
126             velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
127             merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
128             min_window_duration=float(idx.get("min_action_window_duration",

```

```

SERVED.min_window_duration)),
129         )
130
131
132     def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
133         """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
134         (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
135         if obj is None:
136             return {}
137         if isinstance(obj, Mapping):
138             sub = obj.get(key, {})
139         elif hasattr(obj, "get"):
140             sub = obj.get(key, {})
141         else:
142             sub = getattr(obj, key, {})
143         return sub if isinstance(sub, Mapping) else {}
144
145
146     def build_windows(points: List[Any], fps: float, frame_width: float,
147                       preset: WindowingPreset) -> List[Dict[str, Any]]:
148         """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
149
150         A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
151         instead of three loose floats, so eval candidate-building and the served path
reference the
152         SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
153         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
154
155         vt, mg, mwd = preset.as_tuple()
156         return TrackNetRunner.trajectory_to_action_windows(
157             points, fps=fps, frame_width=frame_width,
158             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
159
160
161     def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
162         """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
163         return [(w["start"], w["end"]) for w in windows]
164

```

14. user (2026-06-16T03:12:12.104Z)

```

1     """Promotion gate with the eval?serve guard: a cue is promotable ONLY if it does not
regress
2     at the SERVED windowing.
3
4     **Why this exists.** `per_user_gate.should_promote_for_user` already grades a B?A lift
5     honestly (min-N floor + ?F1 CI excludes 0 + paired sign test). But it grades whatever
scores
6     it is handed ? and the foundational bug was that those scores came from the *recall-
oriented
7     PROPOSAL* windowing (`backend.eval.windowing.PROPOSAL`), which production never
serves. So a
8     cue could clear the bar on a windowing nobody runs (Gate-A: proposal ?F1 **+0.074**)
while
9     regressing on the served one (**?0.008**). Grading harder isn't the fix; **measuring on
the
10    served windowing** is.
11
12    This module composes ? it adds no new statistics. It layers ONE rule on top of

```

```

13     ``should_promote_for_user``:
14
15         A cue is promotable only if, at the **SERVED** windowing, it WINS or at least DOES
NOT
16         REGRESS (its served paired ?F1 mean ? ``-served_regress_eps``, and ? when its served
CI is
17         estimable ? that CI's lower bound does not sit clearly in regression territory).
18
19         A proposal-windowing win is still *reported* (it's useful signal for where to look
next), but it
20         can never carry a promotion on its own. The decision returns BOTH the served and the
proposal
21         verdicts so the eval?serve contrast is explicit and auditable.
22
23         Pure, deterministic, stdlib-only over ``eval_stats``/``per_user_gate``. Additive:
existing eval
24         paths are untouched until a caller routes its A/B through
:func:`promote_if_served_safe`.
25         """
26         from __future__ import annotations
27
28         from dataclasses import dataclass, field
29         from typing import Any, Dict, Optional, Sequence
30
31         from backend.eval.eval_stats import paired_delta_ci
32         from backend.eval.per_user_gate import PromotionDecision, should_promote_for_user
33
34         __all__ = ["ServedGateResult", "promote_if_served_safe", "served_regresses"]
35
36
37         def served_regresses(a_scores: Sequence[float], b_scores: Sequence[float], *,
38                             eps: float = 0.0, n_boot: int = 2000, seed: int = 0) -> Dict[str,
Any]:
39             """Does variant B REGRESS variant A at the served windowing? (the eval?serve guard
core).
40
41             ``a_scores``/``b_scores`` are per-video held-out scores **measured at the SERVED
windowing**
42             (aligned by video). Returns the paired B?A summary (mean ?F1 + bootstrap CI + sign
test) plus
43             a single ``regresses`` boolean:
44
45             - ``regresses=True`` when the served mean ?F1 is below ``-eps`` (a real served
drop), OR
46             the served ?F1 CI lies **entirely** below ``-eps`` (a confidently-negative
band).
47             - ``regresses=False`` when the cue holds the line at the served operating point
(mean ?
48             ``-eps`` and the CI is not a confident net-negative).
49
50             ``eps`` is the regression tolerance: 0.0 = "must not drop at all on the served
mean". A tiny
51             positive ``eps`` lets noise-level wobble through (e.g. 0.005 ? half an F1 point at
n?6).
52             """
53             delta = paired_delta_ci(a_scores, b_scores, n_boot=n_boot, seed=seed)
54             mean_delta = float(delta["mean_delta"])
55             ci_low = float(delta["ci_low"])
56             ci_high = float(delta["ci_high"])
57             # Regression if the served mean dropped beyond tolerance, OR the whole CI is net-
negative.
58             mean_regresses = mean_delta < -eps
59             ci_confidently_negative = ci_high < -eps
60             regresses = bool(mean_regresses or ci_confidently_negative)
61             return {

```

```

62         "regresses": regresses,
63         "mean_delta": mean_delta,
64         "ci_low": ci_low,
65         "ci_high": ci_high,
66         "ci_excludes_zero": bool(delta["ci_excludes_zero"]),
67         "sign_test": delta["sign_test"],
68         "n": int(delta["n"]),
69         "eps": float(eps),
70     }
71
72
73     @dataclass
74     class ServedGateResult:
75         """A promotion decision that BOTH measures the cue on the served windowing AND
reports the
76         proposal-windowing view, so the eval?serve contrast is never hidden.
77
78         - ``promote`` ? final verdict. ``True`` only when the underlying honest gate would
promote
79         *and* the served check did not veto (the cue does not regress at the served
windowing).
80         - ``served_safe`` ? the eval?serve guard's verdict in isolation (cue does not
regress served).
81         - ``vetoed_by_served`` ? ``True`` when the proposal/base gate WOULD have promoted
but the
82         served regression check overruled it (the exact Gate-A failure mode this module
prevents).
83         - ``base_decision`` ? the ``PromotionDecision`` from ``should_promote_for_user`` on
the
84         *promotion-grade* scores (served by default ? see :func:`promote_if_served_safe`).
85         - ``served`` / ``proposal`` ? the paired B?A summary at each windowing (means + CIs
+ sign
86         tests), so a reviewer sees "+0.074 proposal / ?0.008 served" side by side.
87         - ``reason`` ? short human-readable explanation of the final verdict.
88         """
89
90         promote: bool
91         served_safe: bool
92         vetoed_by_served: bool
93         reason: str
94         base_decision: PromotionDecision
95         served: Dict[str, Any]
96         proposal: Optional[Dict[str, Any]] = field(default=None)
97
98         def as_dict(self) -> Dict[str, Any]:
99             return {
100                 "promote": self.promote,
101                 "served_safe": self.served_safe,
102                 "vetoed_by_served": self.vetoed_by_served,
103                 "reason": self.reason,
104                 "base_decision": self.base_decision.as_dict(),
105                 "served": self.served,
106                 "proposal": self.proposal,
107             }
108
109
110     def promote_if_served_safe(
111         served_a: Sequence[float],
112         served_b: Sequence[float],
113         *,
114         proposal_a: Optional[Sequence[float]] = None,
115         proposal_b: Optional[Sequence[float]] = None,
116         structural: bool = False,
117         min_n: int = 5,
118         p_threshold: float = 0.05,

```

```

119         served_regress_eps: float = 0.0,
120         n_boot: int = 2000,
121         seed: int = 0,
122     ) -> ServedGateResult:
123         """Promote a cue ONLY if it does not regress at the SERVED windowing (the
foundational rule).
124
125         The honest bar (`should_promote_for_user`: min-N floor + ?F1 CI excludes 0 + sign
test) is
126         applied to the **served** scores ? the operating point production runs ? so a
proposal-only
127         win can never carry a promotion. The optional `proposal_` scores are scored too
and
128         reported alongside, purely to surface the eval?serve contrast (they NEVER promote on
their
129         own).
130
131         Args:
132             served_a/served_b: per-video held-out scores for variant A (control) and B (the
cue),
133             measured at the **SERVED** windowing. These drive the decision.
134             proposal_a/proposal_b: the same scores at the PROPOSAL windowing (optional).
Reported for
135             contrast; the served regression guard still has the final say.
136             structural: forwarded to `should_promote_for_user` ? a structural guard (e.g.
Gate-A vs
137             held-shuttle FPs) is never *disabled* on "no lift", but it is ALSO not
auto-*promoted* to
138             the served default if it regresses served. So for `structural=True` we report
139             `keep_on` honestly but `promote` reflects the served-safety check.
140             served_regress_eps: served regression tolerance (0.0 = "no served-mean drop
allowed").
141
142         Returns a :class:`ServedGateResult` with the final verdict + both windowings'
evidence.
143         """
144         # 1) The honest gate, applied to the SERVED scores (never the proposal ones).
145         base = should_promote_for_user(
146             served_a, served_b,
147             structural=structural, min_n=min_n, p_threshold=p_threshold,
148             n_boot=n_boot, seed=seed,
149         )
150
151         # 2) The eval?serve guard: does B regress A at the served windowing?
152         served = served_regresses(served_a, served_b, eps=served_regress_eps,
153                                   n_boot=n_boot, seed=seed)
154         served_safe = not served["regresses"]
155
156         # 3) Optional proposal-windowing view (reported, never promoting on its own).
157         proposal: Optional[Dict[str, Any]] = None
158         if proposal_a is not None and proposal_b is not None:
159             proposal = served_regresses(proposal_a, proposal_b, eps=served_regress_eps,
160                                         n_boot=n_boot, seed=seed)
161
162         # 4) Compose the verdict. The served guard can only ever REMOVE a promotion, never
add one.
163         if structural:
164             # A structural guard ships ON regardless of measured lift (base.keep_on stays
True), but
165             # it is only auto-promoted to the SERVED default when it is served-safe. If it
regresses
166             # served, promote=False (don't bake an eval-only win into production) while
keep_on holds.
167             promote = served_safe
168             vetoed = False

```

```

169         if served_safe:
170             reason = ("structural guard kept ON; served-safe (does not regress at the
served "
171                     f"windowing, ?F1 {served['mean_delta']:+.4f}) ? eligible for
served default")
172         else:
173             reason = ("structural guard kept ON, but NOT promoted to the served default:
it "
174                     f"REGRESSES at the served windowing (?F1
{served['mean_delta']:+.4f}); "
175                     "an eval-only win must not be served")
176         else:
177             would_promote = base.promote
178             if would_promote and not served_safe:
179                 promote = False
180                 vetoed = True
181                 reason = ("VETOED by the served-windowing guard: the cue clears the honest
bar but "
182                         f"REGRESSES at the served windowing (served ?F1
{served['mean_delta']:+.4f}")
183                 f", CI [{served['ci_low']:+.4f}, {served['ci_high']:+.4f}]). A win
on a "
184                         "windowing production doesn't serve cannot promote.")
185             elif would_promote:
186                 promote = True
187                 vetoed = False
188                 reason = ("promoted: clears the honest bar AND is served-safe (served ?F1 "
189                         f"{served['mean_delta']:+.4f}). " + base.reason)
190             else:
191                 promote = False
192                 vetoed = False
193                 reason = base.reason
194
195         return ServedGateResult(
196             promote=promote,
197             served_safe=served_safe,
198             vetoed_by_served=vetoed,
199             reason=reason,
200             base_decision=base,
201             served=served,
202             proposal=proposal,
203         )
204

```

15. user (2026-06-16T03:12:12.363Z)

```

1     """SERVED Gate-A verification: run the *production* ``FusionSegmenter`` decision seams
2     on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.
3
4     The offline litmus (``backend.eval.ablation`` / ``fusion_compare``) measures Gate A on
the
5     **permissive proposal windowing** (``distill_local.PROPOSAL`` = recall-oriented) re-
scored
6     through the experiment harness's ``_gate_mask``. That validates the *signal*. It does
NOT
7     exercise the **served path**: in production the windows come from
8     ``trajectory_to_action_windows`` at the production operating point
9     (``velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0``),
the
10    net-crossing count is computed by ``FusionSegmenter._enrich_candidate_stats``, and the
gate is
11    ``FusionSegmenter._select_for_handoff``. THIS module drives exactly those production
methods ?
12    no reimplementations of the windowing or the gate ? on a cached trajectory CSV, so the
13    verification reflects what ``process_video`` actually hands to Gemini.

```

```

14
15 GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that
WOULD
16 be sent to the AI). With the person cue OFF (the default) nothing here loads torch or
touches
17 the GPU ? it just re-scores cached WASB trajectories.
18
19 Run (owner box, golden trajectories present):
20 python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
21 """
22 from __future__ import annotations
23
24 import argparse
25 import json
26 from dataclasses import dataclass
27 from typing import Any, Dict, List, Optional, Tuple
28
29 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30 from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
31 from backend.eval.calibrate_wasb import load_golden_gts
32 from backend.eval.metrics import Interval, score, Score
33 from backend.eval import segmentation_metrics as _segm
34
35 # Production windowing operating point (config.json indexing defaults ? the values
36 # TrajectoryHybridSegmenter.process_video reads for the served fusion run).
37 PROD_VELOCITY_THRESH = 0.02
38 PROD_MERGE_GAP = 2.0
39 PROD_MIN_WINDOW = 2.0
40
41
42 def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
43     """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
44     config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
45     person cue OFF (so no GPU / torch is touched)."""
46     return {
47         "fusion": {
48             "substrate": "wasb",
49             "min_crossings": min_crossings,
50             "min_players": 0,
51             "cue_flags": {},
52         },
53         "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
54         "dead_time_merge_gap": PROD_MERGE_GAP,
55         "min_action_window_duration": PROD_MIN_WINDOW,
56     }
57
58
59 def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
60                             min_crossings: int) -> List[Interval]:
61     """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and
return the
62     windows that would reach the AI handoff (i.e. the served rally candidates).
63
64     Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats``
(computes
65     ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the
production
66     ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue
is OFF.
67     """
68     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
69     windows = TrackNetRunner.trajectory_to_action_windows(
70         points, fps=fps, frame_width=frame_width,
71         velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
72         min_window_duration=PROD_MIN_WINDOW,

```

```

73         )
74         idx_cfg = _served_idx_cfg(min_crossings)
75         # Construct the real served segmenter (db=None, config carries the indexing block).
We only
76         # call the pure decision seems ? no process_video, no DB, no provider ? exactly as
the
77         # registry constructs it for `name`/`description`.
78         seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
79         # The served per-window stats (incl. net_crossings) via the production enrichment
hook.
80         window_stats = [
81             seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
82                                     idx_cfg=idx_cfg)
83             for cw in windows
84         ]
85         keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
86         return [(windows[i]["start"], windows[i]["end"]) for i in keep]
87
88
89     @dataclass
90     class ServedVideoResult:
91         name: str
92         num_gt: int
93         on: Score          # Gate-A ON (min_crossings=2)
94         off: Score         # Gate-A OFF (min_crossings=0, control)
95         seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
96         seg_ratio_off: float
97         n_handoff_on: int
98         n_handoff_off: int
99
100         @property
101         def delta_f1(self) -> float:
102             return self.on.f1 - self.off.f1
103
104
105     def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
106                     on_crossings: int = 2) -> ServedVideoResult:
107         """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff
windows for
108         one golden video against its golden labels."""
109         fps, fw = float(spec["fps"]), float(spec["frame_width"])
110         traj, labels = str(spec["trajectory"]), str(spec["labels"])
111         gts = load_golden_gts(labels)
112         preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
113         preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
114         return ServedVideoResult(
115             name=str(spec["name"]),
116             num_gt=len(gts),
117             on=score(preds_on, gts, iou),
118             off=score(preds_off, gts, iou),
119             seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
120             seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
121             n_handoff_on=len(preds_on),
122             n_handoff_off=len(preds_off),
123         )
124
125
126     def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
127                     on_crossings: int = 2) -> List[ServedVideoResult]:
128         return [evaluate_video(s, iou, on_crossings) for s in specs]
129
130
131     def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
132         """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
133         n = len(results)

```

```

134
135     def _mean(getter) -> float:
136         return sum(getter(r) for r in results) / n if n else 0.0
137
138     deltas = [r.delta_f1 for r in results]
139     wins = sum(1 for d in deltas if d > 1e-9)
140     losses = sum(1 for d in deltas if d < -1e-9)
141     return {
142         "num_videos": n,
143         "mean_f1_on": _mean(lambda r: r.on.f1),
144         "mean_f1_off": _mean(lambda r: r.off.f1),
145         "mean_delta_f1": _mean(lambda r: r.delta_f1),
146         "mean_precision_on": _mean(lambda r: r.on.precision),
147         "mean_precision_off": _mean(lambda r: r.off.precision),
148         "mean_recall_on": _mean(lambda r: r.on.recall),
149         "mean_recall_off": _mean(lambda r: r.off.recall),
150         "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
151         "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
152         "wins": wins,
153         "losses": losses,
154     }
155
156
157     def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) ->
158     str:
159         lines = [
160             f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou})
161             ===",
162             f" {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
163             f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
164         ]
165         for r in results:
166             lines.append(
167                 f" {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
168                 f"{r.delta_f1:>7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
169                 f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
170         lines += [
171             f" {'-' * 92}",
172             f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
173             f"?F1={agg['mean_delta_f1']:+.3f} "
174             f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
175             f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
176             f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
177             f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']}
178             "
179             f"(losses {agg['losses']})",
180         ]
181         return "\n".join(lines)
182
183
184     def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult],
185     Dict[str, Any]]:
186         with open(manifest_path, encoding="utf-8") as f:
187             specs = json.load(f)
188             results = evaluate_manifest(specs, iou=iou)
189             return results, aggregate(results)
190
191
192     def main(argv: Optional[List[str]] = None) -> None:
193         ap = argparse.ArgumentParser(
194             description="Verify served Gate-A on real footage via the production
195             FusionSegmenter path.")
196         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
197         ap.add_argument("--iou", type=float, default=0.5)
198         args = ap.parse_args(argv)

```

```

194         results, agg = run(args.manifest, iou=args.iou)
195         print(format_table(results, agg, args.iou))
196
197
198     if __name__ == "__main__":
199         main()
200

```

16. assistant (2026-06-16T03:12:14.361Z)

Now let me read the remaining critical modules:

17. user (2026-06-16T03:12:15.028Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23    from typing import List, Optional, Tuple
24
25    from backend.eval.annotations import parse_timestamp
26
27    logger = logging.getLogger(__name__)
28
29    Interval = Tuple[float, float]
30
31    _START_NAMES = ("start", "start_time")
32    _END_NAMES = ("end", "end_time")
33
34    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35        """If `row` is a header naming start/end columns, return their indices, else
None."""
36        cells = [(c or "").strip().lower() for c in row]
37        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
38        end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39        if start_idx is not None and end_idx is not None:
40            return start_idx, end_idx
41        return None
42
43
44    def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45        """Load GT rally intervals from either golden-CSV convention.
46
47        strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48        (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.

```

```

49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flatters precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
54     """
55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None    # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                     continue          # consume the header row
70             cols = (0, 1)             # headerless: positional start,end
71             assert cols is not None    # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82             except ValueError as e:
83                 if strict:
84                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                 skipped += 1
86                 continue
87             intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

18. user (2026-06-16T03:12:15.476Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on

```

```

7      CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8      measured by the shipped experiment harness; person features enter only as feature
columns the
9      learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11      Run (on the GPU box):
12      python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14      Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15      `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16      """
17      from __future__ import annotations
18
19      import argparse
20      import json
21      import os
22      import time
23      from typing import Any, Dict, List, Optional
24
25      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26      from backend.pipeline.detectors import rally_gate
27      from backend.pipeline.detectors import fusion_features as ff
28      from backend.eval import distill_local
29      from backend.eval.calibrate_wasb import load_golden_gts
30      from backend.eval.training import label_candidates
31      from backend.utils.run_telemetry import RunTelemetry
32
33      # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34      SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35      # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36      PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39      def _derive_video_path(spec: Dict[str, Any]) -> str:
40          """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41          golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
convention)."""
42          if spec.get("video_path"):
43              return str(spec["video_path"])
44          labels = str(spec["labels"])
45          if labels.endswith(".rallies.csv"):
46              return labels[: -len(".rallies.csv")] + ".MP4"
47          raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
*.rallies.csv")
48
49
50      def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
51                      frame_skip: int = 3, iou: float = 0.5,
52                      log_every_sec: float = 15.0,
53                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
54          """One golden video -> a feature record (shuttle + person features per candidate
window).
55
56          Returns `{name, fps, frame_width,
candidates: [{start, end, shuttle{ }, person{ }, label}], gts}`.
57          `detector` (anything with `detections_over_windows`) is injectable for GPU-free
tests;
58          when None a real `PersonDetector` is built (loads torchvision on first use).
59          `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
60          from the detection progress callback so a hard kill leaves a forensic runlog.

```

```

61     """
62     fps, fw = float(spec["fps"]), float(spec["frame_width"])
63     traj = str(spec["trajectory"])
64     points = TrackNetRunner.parse_trajectory_csv(traj)
65     intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
66     gts = load_golden_gts(str(spec["labels"]))
67     labels = label_candidates(intervals, gts, iou)
68     # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
69     # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
70     axis, net_px, _span = rally_gate.estimate_play_axis(points)
71     video_path = _derive_video_path(spec)
72
73     if detector is None:
74         from backend.pipeline.detectors.person_detector import PersonDetector
75         detector = PersonDetector({"indexing": {"use_gpu": True}})
76
77     # Single-pass person detection over ALL candidate windows (open the video ONCE, read
78     # forward ? no per-window seek). Then compute features per window (pure, no GPU).
79     n = len(intervals)
80     win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
81     print(f"[fusion_golden]   {spec['name']}: {n} windows ? person-detecting in one
pass...", flush=True)
82     t0 = time.monotonic()
83     state = {"last": t0}
84
85     def _progress(done: int, total: int) -> None:
86         if telemetry is not None:
87             telemetry.snapshot(done, total, phase=spec["name"])
88             now = time.monotonic()
89             if now - state["last"] >= log_every_sec or done >= total:
90                 el = now - t0
91                 rate = done / el if el > 0 else 0.0
92                 eta = (total - done) / rate if rate > 0 else 0.0
93                 pct = 100 * done // total if total else 0
94                 print(f"[fusion_golden]   {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96                 state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                             court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),

```

```

121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                                if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141         # break extraction, so guard the build and fall back to no telemetry on any error.
142         telem: Optional[RunTelemetry]
143         try:
144             telem = RunTelemetry(os.path.join(out_dir, "run-telemetry.jsonl"),
label="fusion_golden")
145         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
observes
146             telem = None
147         written: List[str] = []
148         for vi, spec in enumerate(specs, 1):
149             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150             # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior

```

19. user (2026-06-16T03:12:15.699Z)

```

1     """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
set.
2
3     The fusion-P3 measurement. Loads the per-video feature JSONs written by
`fusion_golden.py`
4     and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
out) ?
5     no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
harness's
6     designed seam): the person features are extra columns the learned LogisticRegression
weights,
7     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
8
9     Run (no GPU):
10     python -m backend.eval.fusion_compare --features-dir output --record
11     """
12     from __future__ import annotations
13
14     import argparse
15     import glob
16     import json
17     import os
18     from typing import Any, Dict, List
19

```

```

20     from backend.eval import experiment
21     from backend.eval.eval_stats import paired_delta_ci
22     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
23
24
25     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
26         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
test
27         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
scores
28         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
videos."""
29         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
30         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
31         vids = [p["video"] for p in res["variant_a"]["per_video"]]
32         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
33
34
35     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
36         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
37         feature columns + golden gts). The harness derives labels from gts itself."""
38         videos: List[experiment.VideoCandidates] = []
39         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
40             with open(path, encoding="utf-8") as f:
41                 d = json.load(f)
42                 cand = d["candidates"]
43                 intervals = [(float(c["start"]), float(c["end"])) for c in cand]
44                 features: Dict[str, List[float]] = {}
45                 for fn in SHUTTLE_FEATURE_NAMES:
46                     features[fn] = [float(c["shuttle"][fn]) for c in cand]
47                 for fn in PERSON_FEATURE_NAMES:
48                     features[fn] = [float(c["person"][fn]) for c in cand]
49                 gts = [(float(a), float(b)) for a, b in d["gts"]]
50                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
51                                                         features=features, gts=gts))
52         return videos
53
54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
feature_names=list(SHUTTLE_FEATURE_NAMES),
58                                     threshold=threshold)
59         fusion = experiment.Variant(name="shuttle_person_fusion",
60                                     feature_names=list(SHUTTLE_FEATURE_NAMES) +
list(PERSON_FEATURE_NAMES),
61                                     threshold=threshold)
62         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
63
64
65     def format_result(res: Dict[str, Any], iou: float) -> str:
66         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
67         ds = delta_stats(res)
68         st = ds["sign_test"]
69         lines = [
70             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
===",
71             f"  shuttle-only : F1={a['f1']:.3f}  P={a['precision']:.3f}  R={a['recall']:.3f}
IoU={a['mean_iou']:.3f}",
72             f"  +person      : F1={b['f1']:.3f}  P={b['precision']:.3f}  R={b['recall']:.3f}
IoU={b['mean_iou']:.3f}",
73             f"  ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)
?P={d['precision']:+.3f}  ?R={d['recall']:+.3f}",
74             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "

```

```

75         f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
76         f"| CI clears 0: {ds['ci_excludes_zero']}",
77         " per-video ?F1:",
78     ]
79     for pv in res["per_video_delta"]:
80         lines.append(f"    {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
81     return "\n".join(lines)
82
83
84     def main() -> None:
85         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
shuttle+person (golden set).")
86         ap.add_argument("--features-dir", default="output")
87         ap.add_argument("--iou", type=float, default=0.5)
88         ap.add_argument("--record", action="store_true", help="append the result to the
experiment leaderboard")
89         args = ap.parse_args()
90
91         videos = load_videos(args.features_dir)
92         if len(videos) < 2:
93             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
{len(videos)})")
94         res = compare(videos, iou=args.iou)
95         print(format_result(res, args.iou))
96         if args.record:
97             row = experiment.record_experiment(res)
98             print(f"recorded -> eval_baselines/experiment_leaderboard.json
({row['timestamp']}")
99
100
101     if __name__ == "__main__":
102         main()
103

```

20. user (2026-06-16T03:12:15.845Z)

```

1     """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3     Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4     `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5     window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6     and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7     of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8     (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10    Run (no GPU):
11        python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12    """
13    from __future__ import annotations
14
15    import argparse
16    import glob
17    import json
18    import os
19    from typing import Any, Dict, List
20
21    from backend.eval import experiment
22    from backend.eval.fusion_compare import delta_stats

```

```

23     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES,
_derive_video_path
24     from backend.pipeline.detectors import audio_features as af
25     from backend.pipeline.audio.hit_detector import detect_hits_in_video
26
27
28     def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
29         """Add the 3 audio features to each existing ``<name>_golden_features.json``
window."""
30         with open(manifest_path, encoding="utf-8") as f:
31             specs = json.load(f)
32             written: List[str] = []
33             for spec in specs:
34                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
35                 if not os.path.isfile(jp):
36                     print(f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
first; skipping")
37                     continue
38                 video = _derive_video_path(spec)
39                 print(f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
flush=True)
40                 hits = detect_hits_in_video(video, k=k)
41                 with open(jp, encoding="utf-8") as f:
42                     data = json.load(f)
43                     for c in data["candidates"]:
44                         c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
45                 with open(jp, "w", encoding="utf-8") as f:
46                     json.dump(data, f, indent=2)
47                 print(f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows", flush=True)
48                 written.append(jp)
49             return written
50
51
52     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
53         """Load the feature JSONs into VideoCandidates including the audio columns (requires
`augment` to have run ? raises if a JSON lacks audio)."""
54         videos: List[experiment.VideoCandidates] = []
55         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
56             with open(path, encoding="utf-8") as f:
57                 d = json.load(f)
58                 cands = d["candidates"]
59                 if cands and "audio" not in cands[0]:
60                     raise SystemExit(f"{path} has no audio features ? run `--augment` first")
61                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
62                 features: Dict[str, List[float]] = {}
63                 for fn in SHUTTLE_FEATURE_NAMES:
64                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
65                 for fn in PERSON_FEATURE_NAMES:
66                     features[fn] = [float(c["person"][fn]) for c in cands]
67                 for fn in af.AUDIO_FEATURE_NAMES:
68                     features[fn] = [float(c["audio"][fn]) for c in cands]
69                 gts = [(float(a), float(b)) for a, b in d["gts"]]
70                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
features=features, gts=gts))
71
72     return videos
73
74
75
76     def compare_increment(videos: List[experiment.VideoCandidates], iou: float = 0.5,
threshold: float = 0.5) -> Dict[str, Any]:
77         """Incremental audio lift: shuttle+person vs shuttle+person+audio (honest
LOVO)."""
78         person = experiment.Variant(
79             name="shuttle_person",
80             feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES),

```

```

threshold=threshold)
82     person_audio = experiment.Variant(
83         name="shuttle_person_audio",
84         feature_names=list(SHUTTLE_FEATURE_NAMES) + list(PERSON_FEATURE_NAMES) +
list(af.AUDIO_FEATURE_NAMES),
85         threshold=threshold)
86     return experiment.compare(videos, person, person_audio, iou=iou, honest=True)
87
88
89     def format_result(res: Dict[str, Any], iou: float) -> str:
90         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
91         ds = delta_stats(res)
92         st = ds["sign_test"]
93         lines = [
94             f"=== Fusion P4 incremental AUDIO LOVO (n={res['num_videos']}, IoU>={iou},
honest) ===",
95             f"  shuttle+person          : F1={a['f1']:.3f}  P={a['precision']:.3f}
R={a['recall']:.3f}",
96             f"  shuttle+person+audio : F1={b['f1']:.3f}  P={b['precision']:.3f}
R={b['recall']:.3f}",
97             f"  incremental ?F1 = {d['f1']:+.3f}  ({res['delta_pct']['f1']:+.1f}%)",
98             f"  paired ?F1 {ds['mean_delta']:+.3f}  [95% CI {ds['ci_low']:+.3f},
{ds['ci_high']:+.3f}]  "
99             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f})  "
100            f"| CI clears 0: {ds['ci_excludes_zero']}",

```

21. assistant (2026-06-16T03:12:17.736Z)

Now let me check for regression runners and experiment leaderboard:

22. user (2026-06-16T03:12:18.439Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth  ? AnnotationProvider (file / vendor / oracle)  [backend.annotations]
8     predictions   ? the production pipeline's stored DB rows    [backend.eval.harness]
9     comparison    ? interval matching + P/R/F1                  [backend.eval.metrics]
10    baseline      ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import hashlib
20    import logging
21    from dataclasses import dataclass, asdict
22    from typing import Dict, List, Optional
23
24    from backend.infrastructure.database import Database
25    from backend.eval import metrics
26    from backend.eval.harness import get_predictions
27    from backend.annotations import annotation_registry
28
29    logger = logging.getLogger(__name__)
30
31    # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32    # actually load a committed baseline to compare against.
33    DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")

```

```

34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
detected
88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }

```

```

96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105            stage: str = "final", iou_threshold: float = 0.5,
106            detector: Optional[str] = None,
107            tolerance: float = DEFAULT_TOLERANCE,
108            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150

```

```

151     return RegressionResult(
152         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153         precision=s.precision, recall=s.recall, f1=s.f1,
154         baseline_precision=base_p, baseline_recall=base_r,
155         tolerance=tolerance, regressed=regressed, reasons=reasons,
156         has_baseline=has_baseline, gt_hash=fp,
157     )
158
159
160 def regress_with_provider(db: Database, video_id: str, video_ref: str,
161     tier: str = "file", stage: str = "final",
162     iou_threshold: float = 0.5,
163     detector: Optional[str] = None,
164     tolerance: float = DEFAULT_TOLERANCE,
165     baseline_path: str = DEFAULT_BASELINE_PATH,
166     guideline: Optional[str] = None,
167     sport: str = "badminton",
168     provider_kwargs: Optional[dict] = None) -> RegressionResult:
169     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172     once those providers land). `video_ref` is whatever that provider expects (a CSV
173     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174     predictions in the DB (the file MD5).
175     """
176     provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179         detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182 def format_result(r: RegressionResult) -> str:
183     verdict = "REGRESSED" if r.regressed else "ok"
184     head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185         f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186     if r.baseline_precision is not None:
187         head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188     return head + ".join(f'\n    - {why}' for why in r.reasons)
189

```

23. user (2026-06-16T03:12:18.879Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """
21        a_start, a_end = a

```

```

22     b_start, b_end = b
23     if a_end <= a_start or b_end <= b_start:
24         return 0.0
25     inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26     if inter <= 0.0:
27         return 0.0
28     union = (a_end - a_start) + (b_end - b_start) - inter
29     return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34     """One matched predicted?ground-truth pair."""
35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
(misses)
46
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                    iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65     # Highest IoU first; ties broken by indices for determinism.
66     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass

```

```

85     class Score:
86         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87         iou_threshold: float
88         num_pred: int
89         num_gt: int
90         true_positives: int
91         false_positives: int
92         false_negatives: int
93         precision: float
94         recall: float
95         f1: float
96         mean_iou: float          # over matched pairs (0.0 if none)
97         mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98         mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100        def as_dict(self) -> dict:
101            return self.__dict__.copy()
102
103
104    def score(preds: List[Interval], gts: List[Interval],
105            iou_threshold: float = 0.5,
106            match_result: Optional[MatchResult] = None) -> Score:
107        """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110        extracting the failure lists); otherwise matching is done here.
111        """
112        mr = match_result if match_result is not None else match_intervals(preds, gts,
113        iou_threshold)
114
115        tp = len(mr.matches)
116        fp = len(mr.unmatched_pred_indices)
117        fn = len(mr.unmatched_gt_indices)
118
119        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122        else 0.0
123
124        if mr.matches:
125            mean_iou = sum(m.iou for m in mr.matches) / tp
126            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127        mr.matches) / tp
128            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129        mr.matches) / tp
130        else:
131            mean_iou = mean_start_err = mean_end_err = 0.0
132
133        return Score(
134            iou_threshold=iou_threshold,
135            num_pred=len(preds), num_gt=len(gts),
136            true_positives=tp, false_positives=fp, false_negatives=fn,
137            precision=precision, recall=recall, f1=f1,
138            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139        )
140
141
142    def pr_curve(preds: List[Interval], gts: List[Interval],
143            thresholds: Optional[List[float]] = None) -> List[Score]:
144        """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
145
146        Lets you see strict-vs-lenient behaviour rather than a single operating point.
147        """
148        if thresholds is None:
149            thresholds = [round(0.1 * i, 1) for i in range(1, 10)]

```

```
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147
```

24. user (2026-06-16T03:12:19.597Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/eval_baselines/experiment_leaderboard.json
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/eval_baselines/experiment_leaderboard.json
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/local-noai/eval_baselines/experiment_leaderboard.json
C:\Users\avidu\Projects\badminton-highlight-indexer/eval_baselines/experiment_leaderboard.json
```

25. assistant (2026-06-16T03:12:21.229Z)

Let me check the golden set size and structure:

26. user (2026-06-16T03:12:21.609Z)

```
1      [
2      {
3          "timestamp": "2026-06-14T06:23:51.507393+00:00",
4          "iou": 0.5,
5          "label_set_hash": "50d98ffbc370",
6          "num_videos": 6,
7          "variant_a": {
8              "name": "shuttle_only",
9              "spec_hash": "47c867c36f2f",
10             "aggregate": {
11                 "precision": 0.31807603780927995,
12                 "recall": 0.3101947096042718,
13                 "f1": 0.3068611057861412,
14                 "mean_iou": 0.5791831889148886
15             }
16         },
17         "variant_b": {
18             "name": "shuttle_person_fusion",
19             "spec_hash": "253f4709b4c4",
20             "aggregate": {
21                 "precision": 0.3617228001510626,
22                 "recall": 0.2708939086387243,
23                 "f1": 0.2863398846047612,
24                 "mean_iou": 0.5690111885714347
25             }
26         },
27         "delta": {
28             "precision": 0.04364676234178266,
29             "recall": -0.0393008009655475,
30             "f1": -0.02052122118138,
31             "mean_iou": -0.01017200034345389
32         },
33         "honest_delta_f1": {
34             "mean_delta": -0.02052122118137999,
35             "ci_low": -0.16462806145542527,
36             "ci_high": 0.16224446866060574,
37             "n": 6.0,
38             "ci_excludes_zero": false,
39             "sign_test": {
40                 "wins": 2.0,
41                 "losses": 4.0,
42                 "ties": 0.0,
43                 "n_effective": 6.0,
44                 "p_value": 0.6875
45             }
46         }
47     ]
```

```

46     }
47 },
48 {
49     "timestamp": "2026-06-14T06:24:56.487711+00:00",
50     "iou": 0.5,
51     "label_set_hash": "50d98ffbc370",
52     "num_videos": 6,
53     "variant_a": {
54         "name": "shuttle_person",
55         "spec_hash": "ee89f1e41604",
56         "aggregate": {
57             "precision": 0.3617228001510626,
58             "recall": 0.2708939086387243,
59             "f1": 0.2863398846047612,
60             "mean_iou": 0.5690111885714347
61         }
62     },
63     "variant_b": {
64         "name": "shuttle_person_audio",
65         "spec_hash": "d68c39eba87c",
66         "aggregate": {
67             "precision": 0.4530798177857001,
68             "recall": 0.3408138120839734,
69             "f1": 0.3655826552797436,
70             "mean_iou": 0.7074802852140344
71         }
72     },
73     "delta": {
74         "precision": 0.0913570176346375,
75         "recall": 0.06991990344524907,
76         "f1": 0.07924277067498242,
77         "mean_iou": 0.13846909664259965
78     },
79     "honest_delta_f1": {
80         "mean_delta": 0.07924277067498244,
81         "ci_low": -0.053508315003560535,
82         "ci_high": 0.2029045239922802,
83         "n": 6.0,
84         "ci_excludes_zero": false,
85         "sign_test": {
86             "wins": 4.0,
87             "losses": 2.0,
88             "ties": 0.0,
89             "n_effective": 6.0,
90             "p_value": 0.6875
91         }
92     }
93 },
94 {
95     "timestamp": "2026-06-14T00:00:00+00:00",
96     "iou": 0.5,
97     "label_set_hash": "50d98ffbc370",
98     "num_videos": 6,
99     "promotion": {
100         "cue": "gate_a_net_crossings",

```

27. user (2026-06-16T03:12:21.973Z)

```

total 28
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 .
drwxr-xr-x 1 avidu 197609    0 Jun 16 01:52 ..
-rw-r--r-- 1 avidu 197609 4696 Jun 14 16:28 experiment_leaderboard.json
-rw-r--r-- 1 avidu 197609 1977 Jun 10 17:43 gemini_wrapper_2026-06-10.json
-rw-r--r-- 1 avidu 197609  689 Jun 10 15:57 heuristic_lovo_n6.json

```

28. user (2026-06-16T03:12:22.438Z)

6

29. user (2026-06-16T03:12:22.545Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\output\adarsh_avi_singles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output\gbaaddy_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output\kushagra_singles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output\largetest_doubles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output\mahadevpura_singles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output\testlarge_short_golden_features.json
```

30. assistant (2026-06-16T03:12:24.454Z)

Let me check for more eval modules and the per_user_gate:

31. user (2026-06-16T03:12:24.825Z)

```
1      """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3      The owner adopted personalization as a flag-gated feature on a generalization-
first
4      baseline: a cue/variant B may be flipped ON for one user only when that user's own
5      held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6      fall back to the shipped baseline (variant A). This module is the decision logic for
that
7      flip; it is pure (no I/O) and reuses backend.eval.eval_stats` for every statistic
8      (paired_delta_ci` + paired_sign_test`) so the math is never reinvented.
9
10     The counter-case this guards against (and its mitigations, implemented here):
11
12     1. Minimum-N floor. Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13     (see eval_stats` C1). Refuse to promote below min_n` videos ? point estimates
from
14     too few held-out videos are noise, not signal.
15     2. Honest lift, not a single mean. Promote only when the paired ?F1 95% bootstrap CI
16     excludes 0 and the paired sign test agrees (majority of videos win,  $p \leq$ 
threshold).
17     This is the global promotion bar applied per user.
18     3. Structural guards are never disabled on "no lift". A structural guard (e.g.
Gate-A
19     net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20     modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21     that simply lacks those failure cases must NEVER turn the guard OFF. For
structural=True`
22     the decision is therefore always "keep ON", independent of the measured delta.
23
24     Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25     default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26     threaded through). Nothing here changes existing eval behaviour; callers opt in.
27     """
28     from __future__ import annotations
29
30     from dataclasses import dataclass, field
```

```

31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
43 structural
44         cue that cleared the full bar).
45         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
46 this is
47         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
48 equals
49         ``promote``.
50         - ``reason`` ? short human-readable explanation of the decision.
51         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
52 tests)
53         so a caller can log/audit exactly why a user was or wasn't personalized.
54         """
55
56         promote: bool
57         keep_on: bool
58         reason: str
59         n: int
60         mean_delta: float
61         ci_low: float
62         ci_high: float
63         ci_excludes_zero: bool
64         sign_test: Dict[str, float] = field(default_factory=dict)
65         floor_met: bool = False
66         structural_protected: bool = False
67
68         def as_dict(self) -> Dict[str, Any]:
69             """Plain-dict view (for JSON logging / telemetry)."""
70             return {
71                 "promote": self.promote,
72                 "keep_on": self.keep_on,
73                 "reason": self.reason,
74                 "n": self.n,
75                 "mean_delta": self.mean_delta,
76                 "ci_low": self.ci_low,
77                 "ci_high": self.ci_high,
78                 "ci_excludes_zero": self.ci_excludes_zero,
79                 "sign_test": self.sign_test,
80                 "floor_met": self.floor_met,
81                 "structural_protected": self.structural_protected,
82             }
83
84     def should_promote_for_user(
85         a_scores: Sequence[float],
86         b_scores: Sequence[float],
87         *,
88         structural: bool = False,
89         min_n: int = 5,
90         p_threshold: float = 0.05,
91         confidence: float = 0.95,
92         n_boot: int = 2000,
93         seed: int = 0,
94         eps: float = 0.0,

```

```

92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
scores.
94         ``a_scores`` / ``b_scores`` are the user's per-video held-out F1 (or any score)
for
96         variant A (control / shipped baseline) and variant B (with the cue), aligned by
video
97         (same order). The decision rules (faithful to the module docstring):
98
99         1. ``structural=True`` ? the guard is never disabled on "no lift": return
100         ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
value is
101         in failure modes the user's corpus may not yet contain.
102         2. Otherwise promote iff ALL hold ? the minimum-N floor (``n >= min_n``), the
paired
103         ?F1 bootstrap CI excludes 0, and the paired sign test agrees (majority of
videos
104         win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
105
106         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
``paired_sign_test``);
107         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
108         """
109         n = min(len(a_scores), len(b_scores))
110         floor_met = n >= min_n
111
112         # --- Evidence (reuse eval_stats; never reinvent the statistics)
-----
113         if n == 0:
114             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
115             sign_test: Dict[str, float] = paired_sign_test([], eps)
116             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
117         else:
118             delta = paired_delta_ci(
119                 a_scores, b_scores,
120                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
121             )
122             mean_delta = float(delta["mean_delta"])
123             ci_low = float(delta["ci_low"])
124             ci_high = float(delta["ci_high"])
125             ci_excludes_zero = bool(delta["ci_excludes_zero"])
126             sign_test = delta["sign_test"]
127
128             sign_p = float(sign_test.get("p_value", 1.0))
129             sign_wins = float(sign_test.get("wins", 0.0))
130             sign_losses = float(sign_test.get("losses", 0.0))
131             # The sign test "agrees" with a promotion only if it is significant AND the majority
132             # leans the winning way (more B-wins than B-losses) ? a significant net loss
must
133             # never satisfy the promotion bar.
134             sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
135
136             # --- Rule 1: structural guards are never disabled on "no lift"
-----
137             if structural:
138                 return PromotionDecision(
139                     promote=False,          # nothing to "promote" ? it ships ON by design
140                     keep_on=True,          # ALWAYS kept ON, regardless of measured per-user
lift
141                     reason=(
142                         "structural guard kept ON regardless of measured per-user lift "
143                         "(value is in failure modes this user's corpus may not yet contain)"
144                     ),
145                     n=n,

```

```
146         mean_delta=mean_delta,
147         ci_low=ci_low,
148         ci_high=ci_high,
149         ci_excludes_zero=ci_excludes_zero,
150         sign_test=sign_test,
```

32. user (2026-06-16T03:12:25.055Z)

```
1      """Train/eval split guard (pure-logic core, in-container).
2
3      Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4      and plan M0:
5      - Split unit = match (never clip / file MD5).
6      - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7      - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8      - This would have caught train/eval leakage in training/eval paths.
9      - Cross-repo collector manifest `split` field wiring + wiring into
10        distill_local / run_regression deferred to later PR.
11
12      Usage (once wired):
13          train, eval_ = load_split(manifest)
14          assert_disjoint(train, eval_)
15      """
16      from __future__ import annotations
17
18      import json
19      import os
20      from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23      def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
Tuple[Set[str], Set[str]]:
24          """Load (train_ids, eval_ids) sets keyed by match id.
25
26          manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
have 'match_id'
27          (or 'id' fallback; must be non-empty str for real match key; name/video_id
deliberately not used as they are weak/MD5).
28
29          Raises ValueError on:
30          - non-list manifest after load
31          - no real match-level key (empty/falsy after explicit check)
32          - split label not exactly "train" or "eval"
33
34          The collector 'split' field and cross-repo wiring is deferred; this supports
35          synthetic or pre-split manifests for the pure-logic guard and tests.
36          """
37          if isinstance(manifest, (str, bytes, os.PathLike)):
38              with open(manifest, "r", encoding="utf-8-sig") as f:
39                  raw = json.load(f)
40                  if not isinstance(raw, list):
41                      raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
42                  data = cast(List[Dict[str, Any]], raw)
43              else:
44                  if isinstance(manifest, dict):
45                      raise ValueError(f"manifest must be a list of entries, got
{type(manifest)}")
46                  data = list(manifest)
47                  if not isinstance(data, list):
48                      raise ValueError(f"manifest must be a list of entries, got {type(data)}")
49
50          train_ids: Set[str] = set()
51          eval_ids: Set[str] = set()
52
53          for entry in data:
```

```

54         if not isinstance(entry, dict):
55             continue
56         # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
57         mid = entry.get("match_id")
58         if mid is None or str(mid).strip() == "":
59             mid = entry.get("id")
60         if mid is None or str(mid).strip() == "":
61             raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62         mid = str(mid).strip()
63         sp = str(entry.get("split", "")).strip().lower()
64         if sp not in ("train", "eval"):
65             raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66         if sp == "eval":
67             eval_ids.add(mid)
68         else:
69             train_ids.add(mid)
70
71     return train_ids, eval_ids
72
73
74     def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75         """Raise if any match-id overlap between train and eval sets.
76
77         This would have caught train/eval leakage (e.g. a video used for both
78         training a segmenter and for regression/eval baselines).
79         """
80         t = set(str(x) for x in train_ids)
81         e = set(str(x) for x in eval_ids)
82         overlap = t & e
83         if overlap:
84             raise ValueError(
85                 f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86                 "This would have caught train/eval leakage."
87             )
88

```

33. user (2026-06-16T03:12:25.493Z)

```

__init__.py
ablation.py
annotations.py
batch.py
calibrate_local.py
calibrate_wasb.py
calibration.py
classifier.py
distill_local.py
eval_partial_labels.py
eval_stats.py
experiment.py
export_wasb_segments.py
fusion_audio.py
fusion_compare.py
fusion_golden.py
gemini_refine.py
gt_loader.py
harness.py
metrics.py
per_user_eval.py
per_user_gate.py
per_user_regression.py
promotion.py

```

rally_seq_proto.py
regression.py
score_calibration.py
segmentation_metrics.py
serve_contrast.py
served_gate_a.py
splits.py
training.py
windowing.py

34. assistant (2026-06-16T03:12:29.024Z)

Let me read a few more critical ones:

35. user (2026-06-16T03:12:29.750Z)

```
1      """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3      The offline learned segmenter (ADR-004) must run locally with no external API and
4      no heavy ML stack ? the project already depends on numpy, and sklearn is not
5      installed. This implements just enough: standardized features, balanced-class
6      gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7      model is a small, human-readable, version-controllable file.
8
9      It deliberately stays simple: the rally-window features are few and roughly
10     linearly separable (high player motion + sustained activity => real rally), so a
11     well-regularized linear model is a sensible, inspectable baseline. Swap in a
12     richer model later behind the same fit/predict_proba/save/load interface.
13     """
14
15     import json
16     from typing import List, Optional
17
18     import numpy as np
19
20
21     class LogisticRegression:
22         def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23             self.lr = lr
24             self.epochs = epochs
25             self.l2 = l2
26             self.w: Optional[np.ndarray] = None
27             self.b: float = 0.0
28             self.mean: Optional[np.ndarray] = None
29             self.std: Optional[np.ndarray] = None
30             self.feature_names: Optional[List[str]] = None
31
32         @staticmethod
33         def _sigmoid(z: np.ndarray) -> np.ndarray:
34             # Clip for numerical stability (avoid overflow in exp).
35             return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37         def _standardize(self, X: np.ndarray) -> np.ndarray:
38             return (X - self.mean) / self.std
39
40         def fit(self, X, y, feature_names: Optional[List[str]] = None,
41               class_weight: str = "balanced") -> "LogisticRegression":
42             """Fit with optional balanced class weighting (rally windows are imbalanced
43             ? many spurious candidates per real rally)."""
44             X = np.asarray(X, dtype=float)
45             y = np.asarray(y, dtype=float)
46             if X.ndim != 2 or X.shape[0] == 0:
47                 raise ValueError("X must be a non-empty 2D array")
48             self.feature_names = feature_names
49
```

```

50         self.mean = X.mean(axis=0)
51         self.std = X.std(axis=0)
52         self.std[self.std == 0] = 1.0 # guard constant features
53         Xs = self._standardize(X)
54         n, d = Xs.shape
55
56         if class_weight == "balanced":
57             n_pos = max(1, int(y.sum()))
58             n_neg = max(1, int((1 - y).sum()))
59             w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
60         else:
61             w_pos = w_neg = 1.0
62         sw = np.where(y == 1, w_pos, w_neg)
63
64         self.w = np.zeros(d)
65         self.b = 0.0
66         for _ in range(self.epochs):
67             p = self._sigmoid(Xs @ self.w + self.b)
68             err = (p - y) * sw
69             grad_w = Xs.T @ err / n + self.l2 * self.w
70             grad_b = float(err.mean())
71             self.w -= self.lr * grad_w
72             self.b -= self.lr * grad_b
73         return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None

```

36. user (2026-06-16T03:12:29.930Z)

```

1     """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3     This is the measurement layer for per-video / per-corpus calibration. It answers
4     two questions honestly:
5
6     1. ***"Did precision improve, by how much?"** ? `improvement_report` scores a
7       baseline config vs a tuned config on the same ground truth and reports the
8       before/after metrics and the % delta.
9
10    2. ***"Is that gain real or overfit?"** ? calibration must be measured on data the
11       tuning never saw, otherwise you're just memorising. Two protocols:

```

```

12     - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13       train folds, score on the held-out fold. The **generalization_gap**
14       (train ? test) is the overfit alarm.
15     - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16       pick the config on all *other* videos, score on the held-out video. Reports
17       per-video held-out scores + mean±std. This is the number to trust.
18
19     Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20     List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21     harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22     """
23     from __future__ import annotations
24
25     import statistics as _st
26     from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28     from backend.eval.metrics import score, Score, Interval
29
30     Config = Any                                # opaque to this module (e.g. a thresholds
dict)
31     PredictFn = Callable[[Config], List[Interval]]
32
33
34     def pct_improvement(before: float, after: float) -> float:
35         """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36         if before <= 0:
37             return float("inf") if after > 0 else 0.0
38         return (after - before) / before * 100.0
39
40
41     def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42         """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43         return score(preds, gts, iou_threshold)
44
45
46     def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47         """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48         if n <= 0:
49             return []
50         k = max(2, min(k, n))                # need >=2 folds; cap at n
51         folds = [list(range(i, n, k)) for i in range(k)]    # round-robin ? balanced,
deterministic
52         splits = []
53         for f in range(k):
54             test = folds[f]
55             train = [i for i in range(n) if i not in set(test)]
56             if test and train:
57                 splits.append((train, test))
58         return splits
59
60
61     def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                     metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63         """Pick the config whose predictions maximise `metric` against `gts`."""
64         best_cfg, best_val = None, -1.0
65         for cfg in configs:
66             val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67             if val > best_val:
68                 best_cfg, best_val = cfg, val
69         return best_cfg, best_val
70
71
72     def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],

```

```

73         k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74     """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76     Per fold: choose the best config on the train rallies, score it on the held-out
77     rallies. A large `generalization_gap` (train >> test) means the thresholds are
78     overfitting that video's quirks.
79
80     Default metric is recall on purpose: predictions are global (whole-video

```

37. user (2026-06-16T03:12:30.217Z)

```

1     """Per-user NO-REGRESSION guard ? the missing per-user analogue of `regression.py`.
2
3     The shared golden set has a disciplined no-regression baseline (`regression.py` +
4     ``regression_baseline.json``): a candidate is gated so a global change can never
silently
5     make the shared corpus worse. Personalization has no such rail ? a deliberately-tuned
6     per-user model "has no shared baseline to regress against", so a per-user retrain or a
7     baseline upgrade could quietly make ONE user worse than the model they're already
running.
8     This module builds that missing per-user regression baseline, mirroring the shared
discipline.
9
10    The decision it makes is deliberately narrow and CONSERVATIVE ? it is a guarantee
against
11    regression*, not a promotion decision (that is `per_user_gate.should_promote_for_user`):
12
13    * CONTROL = the user's frozen incumbent ? the variant/config currently active
FOR THAT
14    USER. For a brand-new user with nothing personalized yet, the incumbent simply IS
the
15    shipped baseline. This is the thing we must never regress below.
16    * CANDIDATE = the proposed replacement (a per-user retrain, or a global baseline
upgrade
17    rolled out to this user).
18
19    Both are evaluated on the user's own held-out videos (`experiment.evaluate_variant`,
LOVO-honest
20    for a learned recipe ? no new scoring math), paired by video, and summarised with the
shipped
21    honest statistics (`eval_stats.paired_delta_ci` ? bootstrap ?F1 CI + exact sign test).
Then:
22
23    * regression ? the candidate is significantly worse: the paired ?F1 (candidate ?
control)
24    95% CI excludes 0 on the WRONG side (`ci_high < 0`). ? REJECT, keep the incumbent.
25    * clear improvement ? the candidate is significantly better: the CI excludes 0
on the
26    right side AND the sign test agrees (candidate wins on strictly more videos than it
loses).
27    ? ACCEPT the swap.
28    * tie / insufficient evidence ? the CI spans 0 (the small-N near-null regime):
there is no
29    evidence the swap helps, so we never take the risk. ? KEEP THE INCUMBENT.
30
31    So we only ACCEPT a swap we can defend and only ever toward a measured improvement;
everything
32    ambiguous keeps the incumbent. That is strictly stronger than "don't regress" (it also
refuses
33    risk-neutral churn), which is exactly the safety the personalization feature was
missing.
34
35    A small per-user baseline record is keyed by a `user_id::stage` string (`user_id`
nullable ?

```

```

36     ``None`` = the local single-user deployment; design-for-north-star, implement-for-
current) and
37     saved/loaded as JSON, a sibling to `regression.py`'s ``regression_baseline.json``
approach.
38
39     Pure-CPU, deterministic given ``seed`` (the only randomness is the CI bootstrap); no GPU
/ torch /
40     cv2 / file I/O beyond the explicit baseline save/load. See `docs/EXPERIMENT_HARNESS.md`
+
41     `docs/ANALYZERS_AND_RECONCILERS.md` §13/C1.
42     """
43     from __future__ import annotations
44
45     import json
46     import logging
47     import os
48     from dataclasses import asdict, dataclass, field
49     from typing import Any, Dict, List, Optional, Sequence
50
51     from backend.eval import eval_stats as _stats
52     from backend.eval import experiment
53     from backend.eval.experiment import Variant, VideoCandidates
54
55     logger = logging.getLogger(__name__)
56
57     __all__ = [
58         "DEFAULT_PER_USER_BASELINE_PATH",
59         "PerUserRegressionResult",
60         "PerUserBaseline",
61         "baseline_key",
62         "load_baselines",
63         "save_baseline",
64         "evaluate_no_regression",
65     ]
66
67     # Tracked location (NOT under the gitignored output/), sibling to
68     # regression.DEFAULT_BASELINE_PATH so a fresh checkout / CI can load a committed per-
user
69     # baseline. NULL user_id (local single-user) keys as the literal "local".
70     DEFAULT_PER_USER_BASELINE_PATH = os.path.join("eval_baselines",
"per_user_baseline.json")
71
72
73     # ----- result
74
75     @dataclass
76     class PerUserRegressionResult:
77         """The per-user no-regression decision plus the evidence behind it.
78
79         - ``accept`` ? swap the candidate in for this user (only ever ``True`` on a
*defended*
80         improvement: the ?F1 CI clears 0 on the right side AND the sign test agrees).
81         - ``keep_incumbent`` ? the user keeps their frozen incumbent (control). Always the
exact
82         complement of ``accept`` ? a swap is taken or it is not.
83         - ``regressed`` ? the candidate is *significantly worse* than the incumbent (CI
excludes 0
84         on the wrong side). A True here always implies ``accept=False``.
85         - ``reason`` ? short human-readable explanation, for logging / audit.
86         - the remaining fields surface the paired evidence (per-variant mean held-out F1,
mean ?F1,
87         CI bounds, and the sign test) so a caller can audit exactly why the swap was/
wasn't taken.
88         """
89

```

```

90         accept: bool
91         keep_incumbent: bool
92         reason: str
93         n: int
94         control_f1: float
95         candidate_f1: float
96         mean_delta: float
97         ci_low: float
98         ci_high: float
99         ci_excludes_zero: bool
100        regressed: bool

```

38. assistant (2026-06-16T03:12:33.626Z)

Perfect. Now I have a complete picture. Let me generate the structured inventory:

39. user (2026-06-16T03:12:34.152Z)

Chapter marked: "Eval harness inventory complete". Continue your current work.

40. assistant (2026-06-16T03:14:30.084Z)

Rally-Detection Evaluation + Experiment Harness Inventory

Core Scoring & Metrics

`backend/eval/metrics.py` ? Temporal interval matching

```

- Purpose: Pure, deterministic interval-based metrics (no I/O, no globals).
- Key functions:
  - `interval_iou(a, b)` ? temporal IoU overlap calculation
  - `match_intervals(preds, gts, iou_threshold)` ? greedy one-to-one matching; returns
  `MatchResult` with matches + unmatched indices
  - `score(preds, gts, iou_threshold)` ? full Score: precision, recall, F1, mean_iou, boundary
  errors
  - `pr_curve(preds, gts, thresholds)` ? sweep across IoU thresholds
- Data structures:
  - `Interval = Tuple[float, float]` (seconds)
  - `Score` dataclass: precision, recall, f1, mean_iou, mean_start/end_error
  - `Match`, `MatchResult` for matching output
- Metrics:
  - Precision/recall/F1 (standard definitions)
  - mean_iou (over matched pairs)
  - mean boundary error (start/end timing)
- Gap notes: No F1@k, mAP, or segment-count here (those are in `segmentation_metrics`).

---

```

Segmentation-Aware Metrics

`backend/eval/segmentation\_metrics.py` ? Over-segmentation detection (C4)

```

- Purpose: Spot fragmentation/merging that temporal-IoU masks.
- Key functions:
  - `f1_at_overlaps(preds, gts, overlaps=[0.1, 0.25, 0.5])` ? F1@k at multiple thresholds
  - `average_precision(scored_preds, gts, iou_threshold)` ? AP at one tIoU (VOC-2010
  interpolation)
  - `mean_average_precision(scored_preds, gts, iou_thresholds=[0.3...0.7])` ? mAP@tIoU sweep
  - `segment_count_ratio(preds, gts)` ? len(preds)/len(gts) fragmentation diagnostic
  - `segmentation_report(scored_preds, gts)` ? bundle: F1@k + mAP@tIoU + per-tIoU AP + count
  ratio
- Data structures:
  - `ScoredInterval = Tuple[Interval, float]` (interval + confidence for ranking)
- Metrics:

```

- F1@{10%, 25%, 50%} (temporal-action-localization standard)
- mAP@tIoU (confidence-aware, swept 0.3?0.7)
- segment_count_ratio (crude merge/split detector)
- **Note:** Edit/Levenshtein score intentionally omitted (degenerate for single action class).

Statistical Rigor for Small-N (C1)

`backend/eval/eval_stats.py` ? Bootstrap CIs + paired sign test

- **Purpose:** Honest small-N reporting (??6 videos; avoid single-mean lies).
- **Key functions:**
 - `bootstrap_ci(values, confidence=0.95, n_boot=2000, seed=0)` ? percentile CI deterministic on seed
 - `mean_with_ci(values)` ? {mean, std, n, ci_low, ci_high, confidence}
 - `paired_sign_test(deltas, eps=0)` ? two-sided exact binomial test: wins/losses/ties/p_value
 - `paired_delta_ci(a_values, b_values)` ? B?A summary: mean_delta + CI + sign_test
 - `grouped_folds(groups)` ? leave-one-group-out splits (LOGO, e.g. by match)
 - `out_of_fold_predictions(groups, fit_predict)` ? honest stacking guard (C3): no in-sample preds
- **Outputs:**
 - CI excludes 0 (no overlap zero) ? the promotion bar
 - Paired sign test p-value (how many videos win, not just mean)
 - OOF predictions aligned to input for honest arbiter stacking
- **Scope:** All stats reused by `per_user_gate` + `ablation` + `promotion`.

Per-Cue Score Calibration (C2)

`backend/eval/score_calibration.py` ? Platt + isotonic calibration

- **Purpose:** Map raw cue scores (CNN softmax, audio onset, net-crossing count) to [0,1] probabilities.
- **Key functions:**
 - `fit_platt(scores, labels, iters=200, lr=0.5)` ? parametric: `p=sigmoid(a*s+b)` with target smoothing
 - `fit_isotonic(scores, labels)` ? non-parametric: pool-adjacent-violators (PAVA) fit
 - `brier_score(probs, labels)` ? MSE of probabilities vs 0/1 labels
 - `expected_calibration_error(probs, labels, n_bins=10)` ? ECE across bins
 - `calibration_report(probs, labels)` ? {brier, ece}
- **Data structures:**
 - `PlattCalibrator(a, b)` ? frozen, callable, JSON-serializable
 - `IsotonicCalibrator(xs, ys)` ? piecewise-constant, callable, JSON-serializable
- **Scope:** Calibrators are fitted per LOVO fold on the TRAINING fold only (never held-out), then applied to held-out predictions. Reused by `experiment.predict`.

Variant Definition & Comparison

`backend/eval/experiment.py` ? A/B + LOVO harness (§5 of EXPERIMENT_HARNESS.md)

- **Purpose:** Compose offline re-scoring + LOVO + honest reporting; the thin variant layer over metrics/training.
- **Key classes:**
 - `Variant` ? declarative recipe (name, segmenter, config_overrides, cue_flags, gates, classifier_model, feature_names, threshold, merge_gap, tune_threshold, calibrate)
 - `VideoCandidates` ? one video's persisted detection: intervals + features dict + gts (optional)
 - `CONTROL` ? pinned baseline (no gates, no learned filter, byte-identical)
- **Key public functions:**
 - `predict(variant, vc, model=None, threshold=None, calibrator=None)` ? variant?intervals deterministically
 - `evaluate_variant(videos, variant, iou=0.5, honest=True)` ? per-video + aggregate scores

- ``honest=True`` + learned recipe ? LOVO (train others, predict held-out)
- static variant or pinned model ? no training, direct prediction
- ``compare(videos, variant_a, variant_b, iou=0.5, honest=True, honest_stats=True)`` ? labeled A/B
- Returns: aggregate delta + per-video delta + per-video predictions + label_set_hash (fingerprint)
 - ``honest_stats=True`` (default) adds ``"honest"`` block: variant CIs + paired ?F1 CI + sign test + F1@k + segment-count
 - ``compare_unlabeled(video, variant_a, variant_b, agree_iou=0.5)`` ? side-by-side on unlabeled: agreed/a_only/b_only
 - ``record_experiment(result, path=DEFAULT_LEADERBOARD_PATH)`` ? append JSON row to leaderboard

****How ``compare()`` works:****

1. Evaluate both variants on the labeled corpus (LOVO-honestly for learned recipes).
2. Report per-metric bare-mean deltas.
3. If ``honest_stats=True``, wrap with C1 (bootstrap CIs, paired sign test), C4 (F1@k, segment-count), and the composed ``honest_summary``:
 - ``variant_a_ci`` / ``variant_b_ci`` ? per-metric {mean, std, n, ci_low, ci_high}
 - ``paired_delta_f1`` ? {mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test} (the promotion bar)
 - ``segmentation`` ? {variant_a, variant_b} with F1@{10,25,50}, segment-count ratios
4. The result is the ``"honest"`` block (``experiment_leaderboard.json`` contains these).

****Gates & learned filter:****

- Gate A: ``min_crossings`` ? keep windows where `net_crossings >= threshold`
- Gate B: ``min_players`` ? keep windows where `players_in_court >= threshold` (future)
- Learned filter: supply ``feature_names`` (column list) and optionally a pinned ``classifier_model`` (JSON LogisticRegression)
 - If ``feature_names`` but no ``classifier_model`` and ``honest=True`` ? LOVO trains per-fold
 - ``tune_threshold=True`` ? pick F1-best threshold on the TRAINING fold (C2 / threshold-in-LOVO)
 - ``calibrate="platt"|"isotonic"`` ? fit calibrator on the TRAINING fold, apply to held-out

****Leaderboard location:**** ``eval_baselines/experiment_leaderboard.json`` (git-tracked, survives clean checkout).

Ablation Framework (LOSO / LOGO)

``backend/eval/ablation.py`` ? Leave-one-signal-out marginal contribution

- ****Purpose:**** Measure which features/gates earn their keep via honest small-N contract.
- ****Key classes:****
 - ``Signal`` ? (name, group, columns, gate_knob, structural)
 - ``AblationConfig`` ? (full_variant, signals, granularity, iou, honest, seed, p_threshold)
 - ``AblationRow`` ? one signal's ?(s) + CI + sign test + segment ratios + verdict
 - ``AblationReport`` ? all rows + provenance (granularity, iou, num_videos, full_spec_hash, label_set_hash)
- ****Key functions:****
 - ``ablate_one(videos, cfg, signal)`` ? run ``experiment.compare(ablated_variant, full_variant)``

? AblationRow

- ``run_ablation(videos, cfg)`` ? loop over signals ? AblationReport (sorted by ?F1 desc)
- ``load_videos(features_dir)`` ? load ``*_golden_features.json`` files
- ``default_signals()`` ? 4-group inventory: shuttle (duration, velocity, net_crossings), person (5 cols), audio (3 cols)
- ``litmus_config()`` ? reproduce Gate-A measurement (static `min_crossings=2` vs control)
- ``export_pdf(report, path)`` ? reportlab PDF with table + legend

- ****Verdict vocabulary (no over-claim):****

- ``earns_keep`` ? removing it hurts (CI clears 0 AND sign test agrees)
- ``suggestive`` ? point estimate leans positive but CI spans 0
- ``inert`` ? wash this run, candidate for retirement only after a trend
- ``structural_protected`` ? a guard (Gate-A, failure-mode dependent); never auto-demoted on "no lift"

- ****Scope:**** Pure driver over ``experiment.compare`` + ``eval_stats``; no new scoring math.

Windowing: Eval ? Serve Guard

``backend/eval/windowing.py`` ? Single-source candidate-window presets

- **Purpose:** **THE eval?serve fix** ? decouple evaluation from served windowing; production default is the only authority.
- **Key classes:**
 - ``WindowingPreset`` ? (name, velocity_thresh, merge_gap, min_window_duration)
- **Key presets:**
 - ``SERVED`` ? lifted from ``IndexingConfig()`` defaults (0.02 velocity, 2.0 merge_gap, 2.0 min_window) ? **the production operating point**
 - ``PROPOSAL`` ? recall-oriented permissive (0.005, 1.0, 0.5) ? where the golden feature substrate is built
- **Key functions:**
 - ``served_windowing_from_config(config, family)`` ? extract the actual served windowing from a live config (overrides honored)
 - ``build_windows(points, fps, frame_width, preset)`` ? trajectory?windows under preset
 - ``windows_to_intervals(windows)`` ? normalize to [(start, end)]
- **Critical insight:** Gate-A measured **+0.074** on PROPOSAL but **+0.008** on SERVED ? this module ensures the two can never silently diverge again.

Promotion Gate with Eval?Serve Guard

``backend/eval/promotion.py`` ? Production-safe cue promotion

- **Purpose:** The rule-layer: promote a cue only if it does NOT regress at the SERVED windowing.
- **Key functions:**
 - ``served_regresses(a_scores, b_scores, eps=0, n_boot=2000, seed=0)`` ? paired summary at served windowing; boolean ``regresses`` flag
 - ``promote_if_served_safe(served_a, served_b, proposal_a=None, proposal_b=None, structural=False, min_n=5, p_threshold=0.05, served_regress_eps=0.0)`` ? compose the gate
- **Key class:**
 - ``ServedGateResult`` ? (promote, served_safe, vetoed_by_served, reason, base_decision, served, proposal)
- **Logic:**
 1. Apply ``should_promote_for_user`` to the **SERVED** scores (not proposal).
 2. Check: does B regress A at served windowing?
 3. If would-promote but served-regresses ? veto, return `promote=False, vetoed_by_served=True`.
 4. Optional proposal windowing is reported for contrast, never decides.
 5. Structural guards are reported ``keep_on=True`` but only promoted if ``served_safe=True``.
- **Output:** Both windowings visible side-by-side (the "honest" eval?serve contrast).

Per-User Promotion Gate

``backend/eval/per_user_gate.py`` ? Flag-gated personalization safety

- **Purpose:** Promote a cue per-user only if that user's held-out evidence clears the global bar.
- **Key class:**
 - ``PromotionDecision`` ? (promote, keep_on, reason, n, mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test, floor_met, structural_protected)
- **Key function:**
 - ``should_promote_for_user(a_scores, b_scores, structural=False, min_n=5, p_threshold=0.05)`` ? three rules:
 1. Min-N floor: $n \geq \text{min_n}$ videos
 2. Honest lift: paired ? F1 CI excludes 0 AND sign test agrees
 3. Structural guards: `keep_on=True` always (value in failure modes user corpus may lack)
- **Scope:** Per-user analogue of global promotion gate; reuses ``eval_stats`` for all statistics.

Per-User No-Regression Guard

``backend/eval/per_user_regression.py`` ? Per-user baseline regression check

- **Purpose:** Never silently make one user worse than their frozen incumbent.
- **Key class:**
 - ``PerUserRegressionResult`` ? (accept, keep_incumbent, regressed, reason, n, control_f1, candidate_f1, mean_delta, ci_low, ci_high, sign_test)
- **Key functions:**
 - ``evaluate_no_regression(control_videos, candidate_videos, variant_control, variant_candidate, iou=0.5)`` ? result
 - ``load_baselines(path)`` / ``save_baseline(user_id, stage, control_f1, candidate_f1, path)`` ? per-user baseline JSON
- **Logic:**
 - Regression: candidate ?F1 CI excludes 0 on the **wrong side** (ci_high < 0).
 - Clear improvement: CI excludes 0 on the right AND sign test agrees.
 - Tie/insufficient: CI spans 0 ? keep incumbent.
 - Only accept defended improvements; ambiguous keeps incumbent.
- **Baseline location:** ``eval_baselines/per_user_baseline.json``.

Served Gate-A Verification

``backend/eval/served_gate_a.py`` ? Production path smoke-test

- **Purpose:** Run the actual served FusionSegmenter decision seams on cached trajectories; verify Gate-A on the served windowing (not offline proposal).
- **Key functions:**
 - ``served_handoff_windows(trajecory_csv, fps, frame_width, min_crossings)`` ? run production FusionSegmenter.``_enrich_candidate_stats + ._select_for_handoff`` on cached trajectory
 - ``evaluate_video(spec, iou=0.5, on_crossings=2)`` ? ``ServedVideoResult`` (on/off F1, segment ratio, num handoff)
 - ``evaluate_manifest(specs)`` ? list of results
 - ``aggregate(results)`` ? mean F1 on/off, ?F1, precision, recall, wins/losses
 - ``format_table(results, agg, iou)`` ? ASCII table
 - ``run(manifest_path, iou=0.5)`` ? (results, aggregate)
- **Key class:**
 - ``ServedVideoResult`` ? (name, num_gt, on/off Scores, seg_ratio_on/off, n_handoff_on/off, delta_f1 property)
- **Scope:** GPU-free (no person cue), Gemini-free; stops at handoff boundary. The litmus for "does this gate actually work in production?"

Ground-Truth Loader

``backend/eval/gt_loader.py`` ? Canonical GT interval reader (ADR-008 prereq)

- **Purpose:** One loader that accepts BOTH legacy formats (positional start/end vs named start_time/end_time).
- **Key function:**
 - ``load_gt_intervals(csv_path, strict=True)`` ? loads either convention (header-mapped or positional)
 - Headers: ``start``/``start_time``, ``end``/``end_time``
 - Timestamp format: decimal seconds OR MM:SS / HH:MM:SS
 - ``strict=True`` ? raises on bad rows; ``strict=False`` ? skips, logs count
 - Returns: sorted intervals, warns on overlaps
- **Scope:** Replaces both ``annotations.load_annotations`` and ``calibrate_wasb.load_golden_gts``; both now delegate here.

Golden Feature Substrate

`backend/eval/fusion\_golden.py` ? GPU step: person features on golden set

```
- **Purpose:** One-shot person-detection run over golden videos; writes replayable feature JSONs
(no re-detect for ablation/compare).
- **Key function:**
  - `extract_video(spec, detector=None, frame_skip=3, iou=0.5)` ? {name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}
  - `run(manifest_path, out_dir, frame_skip=3, iou=0.5, fresh=False, smallest_first=False)` ?
list of written paths
- **Output:** `output/<name>_golden_features.json` per video (replayable substrate for
ablation/compare).
- **Feature columns:**
  - Shuttle: duration, peak_velocity, mean_velocity, active_ratio, net_crossings
  - Person: players_in_court, two_sided_motion, mean_player_motion, in_court_ratio,
shuttle_player_colocation
  - Audio (optional, added by fusion_audio): audio_hit_count, audio_hit_rate,
audio_hit_strength_mean
- **Scope:** Mirrors distill_local flow but adds person cue (GPU once, CPU thereafter).

---
```

`backend/eval/fusion\_compare.py` ? CPU LOVO: shuttle-only vs shuttle+person

```
- **Purpose:** Measure person-cue lift on golden set (honest LOVO).
- **Key functions:**
  - `load_videos(features_dir)` ? list of VideoCandidates with 10 feature columns
  - `compare(videos, iou=0.5, threshold=0.5)` ? `experiment.compare(shuttle_only,
shuttle_person_fusion)` result
  - `delta_stats(res, metric="f1")` ? paired ?F1 + CI + sign test (reuses eval_stats)
  - `format_result(res, iou)` ? per-video + aggregate output
- **Output:** Experiment leaderboard row (optional `--record`).

---
```

`backend/eval/fusion\_audio.py` ? Audio cue incremental lift (CPU)

```
- **Purpose:** Measure audio-cue lift on top of person (honest LOVO).
- **Key functions:**
  - `augment(manifest_path, features_dir, k=2.5)` ? add 3 audio features to existing feature
JSONs (ffmpeg + numpy, CPU)
  - `load_videos_with_audio(features_dir)` ? load feature JSONs with audio
  - `compare_increment(videos, iou=0.5, threshold=0.5)` ? `experiment.compare(shuttle+person,
shuttle+person+audio)` result
  - `format_result(res, iou)` ? per-video + aggregate output
- **Output:** Experiment leaderboard row (optional `--record`).

---
```

Regression & Baseline

`backend/eval/regression.py` ? Shared golden-set no-regression guard

```
- **Purpose:** Snapshot "known good" per-video numbers; flag regressions (P/R drops beyond
tolerance).
- **Key functions:**
  - `gt_hash(ground_truth)` ? stable 12-char SHA1 hash of sorted GT intervals (detects label
changes)
  - `load_baselines(path)` / `save_baseline(video_id, stage, precision, recall, f1, path,
iou_threshold, detector, gt_fingerprint)` ? per-video baseline JSON
  - `regress(db, video_id, ground_truth, stage="final", iou_threshold=0.5, detector=None,
tolerance=0.05)` ? `RegressionResult` (precision, recall, f1, baseline_*, regressed, reasons,
has_baseline, gt_hash)
  - `regress_with_provider(db, video_id, video_ref, tier="file", ...)` ? end-to-end: obtain GT
from provider, then regress
- **Key class:**
  - `RegressionResult` ? (video_id, stage, iou_threshold, precision, recall, f1,
baseline_precision/recall, tolerance, regressed, reasons, has_baseline, gt_hash)
- **Baseline location:** `eval_baselines/regression_baseline.json` (git-tracked).
```

- ****Guards:**** Incommensurable comparison detection (baseline taken at different IoU/detector/labels).

Calibration (Per-Video Tuning)

`backend/eval/calibration.py` ? Segmenter threshold tuning + generalization gap

- ****Purpose:**** Measure before/after improvement honestly (cross-validate or leave-one-video-out).
- ****Key functions:****
 - ``pct_improvement(before, after)`` ? (after ? before) / before * 100
 - ``evaluate(preds, gts, iou_threshold)`` ? Score
 - ``kfold_indices(n, k)`` ? deterministic k-fold splits
 - ``select_best(predict_fn, configs, gts, metric="f1", iou_threshold)`` ? (best_cfg, best_val)
 - ``cross_validate(predict_fn, configs, gts, k=5, metric="recall", iou_threshold)`` ? k-fold on one video's rallies; reports train/test per fold + generalization_gap
 - ``leave_one_video_out(predict_fn, configs, videos, gts_per_video, metric="recall", iou_threshold)`` ? per-video held-out scores, mean±std
- ****Scope:**** Segmenter-agnostic; callers pass a `predict_fn(config)`. Used for per-video WASB tuning (`distill_local`).

Classifier

`backend/eval/classifier.py` ? Tiny logistic regression (numpy only)

- ****Purpose:**** Dependency-free learned rally filter (no sklearn, JSON-serializable).
- ****Key class:****
 - ``LogisticRegression(lr=0.1, epochs=1000, l2=1e-3)`` ? standardized features, balanced-class SGD, sigmoid output
- ****Key methods:****
 - ``fit(X, y, feature_names=None, class_weight="balanced")`` ? self (normalizes, trains)
 - ``predict_proba(X)`` ? np.ndarray of probabilities
 - ``predict(X, threshold=0.5)`` ? 0/1 labels
 - ``to_dict()`` / ``from_dict(d)`` ? JSON (model + params + mean/std + feature_names)
 - ``save(path)`` / ``load(path)`` ? JSON file
- ****Scope:**** The offline learned segmenter (ADR-004); reused by ``experiment.predict``.

Train/Eval Split Guard

`backend/eval/splits.py` ? Leakage detection (deferred wiring)

- ****Purpose:**** Load train/eval split from manifest; assert disjointness (match-level, not file/MD5).
- ****Key functions:****
 - ``load_split(manifest)`` ? (train_ids, eval_ids) sets by match_id
 - ``assert_disjoint(train_ids, eval_ids)`` ? raises on any overlap
- ****Scope:**** Pure-logic core, in-container; cross-repo collector manifest wiring deferred.

Supporting Modules

`backend/eval/training.py` ? label candidates (LOVO training data)

- Used by `experiment.py` to label candidates during LOVO training.

`backend/eval/annotations.py` ? timestamp parsing

- Used by `gt_loader.py` for flexible timestamp formats.

`backend/eval/harness.py` ? DB-touching: get_predictions

- Used by regression.py to fetch stored predictions.

``backend/eval/gemini_refine.py` ? merge_overlaps`

- Used by experiment.predict to post-filter overlaps.

Experiment Leaderboard

****Location:**** ``eval/baselines/experiment_leaderboard.json`` (git-tracked)

****Structure (one row per experiment):****

````json`

```
{
 "timestamp": "2026-06-14T06:23:51.507393+00:00",
 "iou": 0.5,
 "label_set_hash": "50d98ffbc370",
 "num_videos": 6,
 "variant_a": {"name": "...", "spec_hash": "...", "aggregate": {...}},
 "variant_b": {"name": "...", "spec_hash": "...", "aggregate": {...}},
 "delta": {...},
 "honest_delta_f1": {"mean_delta": ..., "ci_low": ..., "ci_high": ..., "ci_excludes_zero": ...,
 "sign_test": {...}}
}
```

**\*\*Content:\*\*** Each row is a ``compare()`` result, compact and reproducible (variant specs + hashes + label fingerprint).

---

## Golden Set Today

**\*\*Size:\*\*** 6 golden videos (in ``output/``):

- adarsh\_avi\_singles\_golden\_features.json
- gbaaddy\_golden\_features.json
- kushagra\_singles\_golden\_features.json
- largetest\_doubles\_golden\_features.json
- mahadevpura\_singles\_golden\_features.json
- testlarge\_short\_golden\_features.json

**\*\*Format:\*\*** ``{name, fps, frame_width, candidates:[{start,end,shuttle{}},person{},audio{},label]], gts:[[s,e]...]}``

**\*\*Label-set hash:\*\*** ``50d98ffbc370`` (stable fingerprint; changes when labels change).

---

## Exact Answers to Design-Doc Questions

### 1. How is "Variant" defined? How does ``compare()`` work?

**\*\*Variant\*\*** (``experiment.py:76-154``) is a **\*\*declarative re-scoring recipe\*\***, not a code branch:

- ``name``, ``segmenter``, ``config_overrides``, ``cue_flags`` ? provenance
- ``min_crossings``, ``min_players`` ? decision gates
- ``classifier_model`` ? path to pinned LogisticRegression JSON (frozen artifact)
- ``feature_names`` ? list of persisted feature columns (if None, no learned filter)
- ``threshold`` ? classifier probability cutoff
- ``merge_gap`` ? post-filter overlap-merge window (seconds)
- ``tune_threshold`` ? pick F1-best threshold on the TRAINING fold (C2)
- ``calibrate`` ? "platt" or "isotonic" (fitted on TRAINING fold)
- ``is_learned()`` ? True if classifier\_model or feature\_names set
- ``spec_hash()`` ? 12-char stable hash (recipe reproducibility)

- `CONTROL` ? pinned baseline (name="control", all defaults); byte-identical to control variant always

```
`compare(videos, variant_a, variant_b, iou=0.5, honest=True, honest_stats=True)` (lines 403?448):
1. Call `evaluate_variant(videos, variant_a, iou, honest)` ? eval_a
2. Call `evaluate_variant(videos, variant_b, iou, honest)` ? eval_b
3. Compute per-metric bare-mean deltas (`delta = {m: eval_b[m] - eval_a[m] for m in _METRICS}`)
4. Compute per-metric percentage deltas via `pct_improvement`
5. Compute per-video ?F1 (B?A aligned by video name)
6. Compute `label_set_hash` over all GT intervals (detects label changes)
7. Return result dict with variant_a, variant_b, delta, per_video_delta, delta_pct, label_set_hash, iou, num_videos
8. **If `honest_stats=True`** (default), add `"honest"` block via `honest_summary()``
 - `variant_a_ci` / `variant_b_ci` ? per-metric {mean, std, n, ci_low, ci_high, confidence}
 - `paired_delta_f1` ? {mean_delta, ci_low, ci_high, ci_excludes_zero, sign_test with wins/losses/p_value}
 - `segmentation` ? {variant_a, variant_b} each with f1@{10,25,50}, ap_at_tiou, mean_ap, segment_count_ratio
```

\*\*The "honest" stats block\*\* (lines 384?400) is the promotion-grade evidence: ?F1 CI + paired sign test + segmentation. A promotion clears zero CI AND sign test (not bare mean).

## 2. What metrics exist?

\*\*From `metrics.py`:\*\*

- Precision, recall, F1 (standard interval-based)
- mean\_iou (over matched pairs)
- mean\_start\_error, mean\_end\_error (boundary timing)

\*\*From `segmentation\_metrics.py`:\*\*

- F1@k: F1 at each overlap threshold {10%, 25%, 50%} (TAS standard)
- mAP@tIoU: confidence-aware AP swept 0.3?0.7 (action-detection convention)
- segment\_count\_ratio: len(preds)/len(gts) (fragmentation diagnostic)
- (Edit/Levenshtein intentionally omitted; degenerate for single action class)

\*\*From `eval\_stats.py` (statistical wrappers):\*\*

- Bootstrap CI over per-video scores (percentile, deterministic on seed)
- Paired ?F1 CI + exact two-sided sign test (the promotion bar: CI excludes 0 AND sign test agrees)
- Out-of-fold predictions (C3: honest stacking guard)

\*\*From `score\_calibration.py`:\*\*

- Platt scaling calibrator + IsotonicCalibrator (both JSON-serializable)
- Brier score, expected calibration error (ECE)

\*\*NOT yet wired into default reporting:\*\*

- Per-cue calibration (Platt/isotonic) ? fitted but not evaluated post-hoc; could measure ECE/Brier improvements
- Edit/segmental scores (intentionally omitted)
- Confidence-weighted metrics (AP) ? present in `segmentation\_metrics` but not surfaced in core `experiment.compare` (would need scored predictions with confidences)

## 3. How do no-regression / promotion gating work?

\*\*Promotion path:\*\*

1. \*\*`per\_user\_gate.should\_promote\_for\_user(a\_scores, b\_scores, structural=False, min\_n=5, p\_threshold=0.05)`\*\* ? per-user analog of global gate
  - Rule 1: min-N floor ( $n \geq \text{min\_n}$ )
  - Rule 2: honest lift (paired ?F1 CI excludes 0 AND sign test agrees)
  - Rule 3: structural guards (keep\_on=True always, never disable on "no lift")
  - Returns `PromotionDecision(promote, keep\_on, reason, ...)`
2. \*\*`promotion.promote\_if\_served\_safe(served\_a, served\_b, proposal\_a=None, proposal\_b=None, structural=False, ...)`\*\* ? wraps the per-user gate with eval?serve guard

- Apply ``should_promote_for_user`` to **SERVED** scores (the operating point production runs)
- Call ``served_regresses(served_a, served_b, eps=0)`` ? check if B regresses A at served windowing
- If would-promote but served-regresses ? veto, return ``promote=False, vetoed_by_served=True``
- Optional proposal scores are scored and reported for contrast, never decide
- Returns ``ServedGateResult(promote, served_safe, vetoed_by_served, reason, base_decision, served, proposal)``

**No-regression guard:**

- ``per_user_regression.evaluate_no_regression(control_videos, candidate_videos, variant_control, variant_candidate)`` ? per-user regression check
- Evaluate both variants LOVO-honestly on user's own held-out videos
- Pair by video, compute ?F1 CI
- **Regression:** CI excludes 0 on the **wrong side** (`ci_high < 0`)
- **Clear improvement:** CI excludes 0 on the right AND sign test agrees ? accept swap
- **Tie/insufficient:** CI spans 0 ? keep incumbent (conservative)
- Returns ``PerUserRegressionResult(accept, keep_incumbent, regressed, reason, ...)``
- ``regression.regress(db, video_id, ground_truth, stage, iou_threshold, detector, tolerance)`` ? shared global no-regression baseline
- Compare stored predictions against ground truth
- Baseline lookup: ``regression_baseline.json``
- Guards: incommensurable comparison (different IoU/detector/labels)
- Returns ``RegressionResult(regressed, reasons, ...)``

**Windowing guard:**

- ``windowing.py`` single-sources served preset from ``IndexingConfig()`` defaults (0.02 velocity, 2.0 merge\_gap, 2.0 min\_window)
- ``served_gate_a.py`` runs the actual FusionSegmenter decision seams on cached trajectories, verifying the gate works on the **served** windowing (not offline proposal)
- This was the litmus for Gate-A: **+0.074** on proposal, **?0.008** on served (the motivation for promotion.py)

#### 4. What splits are used? Stacking-leakage guards?

**Splits:**

- **LOVO (leave-one-video-out):** The canonical split. ``evaluate_variant(videos, variant, honest=True)`` trains on all-but-one, predicts the held-out one, repeats for each video.
  - Only for learned variants (`classifier_model=None`, `feature_names` set).
  - Static variants (no learned filter) have no leakage ? direct per-video prediction.
- **LOGO (leave-one-group-out):** From ``eval_stats.grouped_folds(groups)``. Splits by match (or any grouping). Used for honest stacking (C3).
- **k-fold:** From ``calibration.kfold_indices(n, k)``. Per-video tuning (cross-validation).

**Stacking-leakage guards:**

- **C3 ? Honest stacking:** ``eval_stats.out_of_fold_predictions(groups, fit_predict)`` ? ensures a learned producer's outputs, when fed as arbiter features, are out-of-fold (never in-sample).
  - Implemented but **not yet wired into default reporting** (no arbiter in the current harness).
- **Per-LOVO-fold calibration/threshold tuning:** In ``experiment._calibrate_and_tune(variant, model, train_videos, iou)`` ? calibrator and threshold are fitted on the **TRAINING fold only**, never the held-out one. Lines 243?275.

**No cross-contamination in LOVO:** Each fold isolates one video as held-out; training uses all others. Labels are per-video, so no label leakage even across folds.

#### 5. How are golden labels loaded? On-disk format? How many golden videos today?

**Golden label loader:** ``gt_loader.load_gt_intervals(csv_path, strict=True)``

- Accepts **both** legacy conventions:
  1. Header-mapped: ``start`/`start_time`` and ``end`/`end_time`` columns (rally-annotator golden format)
  2. Positional: columns 0, 1 (collector's curate-export format)
- Timestamp format: decimal seconds OR MM:SS / HH:MM:SS (parsed by ``annotations.parse_timestamp``)
- ``strict=True`` ? raises on bad rows (corpus/gate use)

- `strict=False` ? skips bad rows, logs count (legacy tuning behavior)
- Returns: sorted intervals, warns on overlaps (labeling error detector)
- This is **THE canonical one loader** (replaces both `annotations.load\_annotations` and `calibrate\_wasb.load\_golden\_gts`).

**Golden feature substrate on-disk format:**

- **File:** `output/<name>\_golden\_features.json` (written by `fusion\_golden.py`)
- **JSON schema:**

```

{
 "name": "adarsh_avi_singles",
 "fps": 30.0,
 "frame_width": 1280.0,
 "candidates": [
 {
 "start": 1.5, "end": 3.2,
 "shuttle": {"duration": 1.7, "peak_velocity": 0.95, ...},
 "person": {"players_in_court": 2, "two_sided_motion": 0.8, ...},
 "audio": {"audio_hit_count": 5, "audio_hit_rate": 2.9, ...}, // optional
 "label": 1 // 0 or 1 (GT vs non-GT)
 },
 ...
],
 "gts": [[1.5, 3.2], [5.1, 7.8], ...] // ground-truth rally intervals
}

```
- **Feature columns:**
  - Shuttle (5): duration, peak\_velocity, mean\_velocity, active\_ratio, net\_crossings
  - Person (5): players\_in\_court, two\_sided\_motion, mean\_player\_motion, in\_court\_ratio, shuttle\_player\_colocation
  - Audio (3, optional): audio\_hit\_count, audio\_hit\_rate, audio\_hit\_strength\_mean

**Golden videos today:** **6 videos** (stable corpus)

1. adarsh\_avi\_singles\_golden\_features.json
2. gbaaddy\_golden\_features.json
3. kushagra\_singles\_golden\_features.json
4. largetest\_doubles\_golden\_features.json
5. mahadevpura\_singles\_golden\_features.json
6. testlarge\_short\_golden\_features.json

**Label-set fingerprint:** `50d98ffbc370` (stable; detects when labels change). Carried by every experiment leaderboard row.

---

## Gaps & "Not Wired Into Default Reporting"

1. **Per-cue calibration ECE/Brier measurements** ? calibrators are fitted (Platt/isotonic) but not evaluated post-hoc; could measure whether calibration *improved* match confidence.
2. **Confidence-weighted metrics (AP)** ? `segmentation\_metrics` implements `average\_precision` + `mean\_average\_precision`, but core `experiment.compare` doesn't surface them (would need scored predictions with confidences per candidate).
3. **Out-of-fold stacking** (C3) ? mechanism exists (`eval\_stats.out\_of\_fold\_predictions`) but no arbiter in the harness yet to use OOF features honestly.
4. **Edit/segmental scores** ? intentionally omitted (degenerate for single-class temporal matching).
5. **Per-window confidence scores from detection** ? features are raw signals (shuttle velocity, person count, audio hits), not probabilities; no learned confidence from the detector pipeline per-candidate window (only from the learned arbiter).

---

## Summary

The eval harness is **complete and composable**:

- **Scoring**: ``metrics` + `segmentation_metrics``
- **Stats**: ``eval_stats`` (C1: CIs + sign test), ``score_calibration`` (C2: Platt/isotonic), ``ablation`` (C4: segment-count)
- **Variants & A/B**: ``experiment`` (compare, predict, evaluate\_variant, record leaderboard)
- **Gates & gating**: ``per_user_gate``, ``promotion`` (with eval?serve guard), ``per_user_regression``, ``served_gate_a`` verification
- **Golden set**: ``fusion_golden``, ``fusion_compare``, ``fusion_audio`` (GPU?CPU pipeline)
- **Baseline**: ``regression`` (shared), ``per_user_regression`` (per-user)
- **Windowing**: ``windowing`` (single-sourced presets; eval?serve fix)

**The design doc can extend this machinery** (e.g., add per-cue calibration measurement, wire OOF stacking into an arbiter, surface AP confidence metrics, add cross-video LOGO splits), but it **does not need to reinvent** any core scoring, matching, or statistical foundations ? they are all present and tested.